

Assignment 7 - Bank Database

Branch, Customer, Loan_Application with ternary relationship.

Commands :

```
DROP TABLE IF EXISTS ternary, loan_application,
customer, branch CASCADE; CREATE TABLE branch (
    bid INT PRIMARY KEY,
    brname VARCHAR(30) NOT
    NULL UNIQUE, brcity
    VARCHAR(20) NOT NULL
);
CREATE TABLE
    customer ( cno
    INT PRIMARY
    KEY,
    cname VARCHAR(20)
    NOT NULL, caddr
    VARCHAR(35),
    city VARCHAR(20) NOT NULL
);
CREATE TABLE
    loan_application (
    lno INT PRIMARY
    KEY,
    lamtrequired NUMERIC(12,2) NOT NULL CHECK (lamtrequired > 0),
    lamtapproved NUMERIC(12,2) NOT NULL CHECK
    (lamtapproved >
    0)
    , ldate DATE NOT NULL
);
CREATE TABLE ternary (
    bid INT REFERENCES
    branch(bid), cno INT
    REFERENCES
    customer(cno),
    lno INT REFERENCES
    loan_application(lno), PRIMARY
    KEY (bid, cno, lno)
);
INSERT INTO branch VALUES
(1,'Aundh','Pune'),
(2,'Pimpri','Pune'),
(3,'Deccan','Pune'),
(4,'M.G. ROAD','Pune'),
(5,'J.M.Road','Pune');
INSERT INTO customer
VALUES (101,'Asha','Karve
Nagar','Pune'),
(102,'Amit','Baner','Pune'),
(103,'Pooja','Nigdi','Pune'),
(104,'Rohit','Camp','Pune'),
```

```

(105,'Anita','Aundh','Pune'),
(106,'Mohan','Hadapsar','Pune'),
(107,'Mira','Wakad','Pune'),
(108,'Sonal','Kothrud','Pune'),
(109,'Ajay','Shivajinagar','Pune');
INSERT INTO
loan_application VALUES
(1001,500000,450000,'2019-
06-10'),
(1002,800000,800000,'2020-01-15'),
(1003,300000,250000,'2020-05-14'),
(1004,1200000,1000000,'2021-04-12'),
(1005,150000,100000,'2019-12-12'),
(1006,250000,250000,'2021-04-12'),
(1007,900000,850000,'2020-07-10'),
(1008,100000,100000,'2020-03-21'),
(1009,400000,350000,'2021-01-05'),
(1010,220000,200000,'2020-09-25'),
(1011,175000,150000,'2021-03-03');
INSERT INTO ternary VALUES
(1,101,1001),
(2,102,1002),
(4,103,1003),
(3,104,1004),
(1,105,1005),
(4,106,1006),
(3,102,1007),
(5,103,1008),
(4,107,1009),
(5,108,1010),
(5,109,1011);
SELECT b.brname, SUM(l.lamtapproved) AS
total_approved FROM branch b
JOIN ternary t ON b.bid = t.bid
JOIN loan_application l ON t.lno = l.lno
WHERE l.date BETWEEN '2019-06-01' AND '2020-06-01'
GROUP BY
b.brname ORDER
BY b.brname;
SELECT DISTINCT c.*
FROM customer c
JOIN ternary t ON
c.cno = t.cno JOIN
branch b ON b.bid =
t.bid WHERE b.brname
= 'Aundh';
SELECT SUM(lamtrequired) AS
total_required FROM
loan_application;
SELECT * FROM branch;
CREATE OR REPLACE FUNCTION
trg_validate_customer() RETURNS TRIGGER
AS $$
BEGIN

```

```
IF NEW.cno < 0 OR NEW.cname IS NULL THEN
    RAISE EXCEPTION 'Customer number must be positive and customer name
    cannot be null';
```

```
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER
before_insert_customer BEFORE
INSERT ON customer
FOR EACH ROW
EXECUTE FUNCTION trg_validate_customer();
```

```
CREATE OR REPLACE FUNCTION
fn_customer_loans(p_customer_name VARCHAR) RETURNS
TABLE (
    loan_no INT,
    required_amount
    NUMERIC(12,2),
    approved_amount
    NUMERIC(12,2),    loan_date
    DATE,
    branch_name VARCHAR(30)
) AS $$
BEGIN
    RETURN QUERY
    SELECT l.lno, l.lamtrequired, l.lamtapproved, l.ldate,
    b.brname FROM customer c
    JOIN ternary t ON c.cno = t.cno
    JOIN loan_application l ON l.lno
    = t.lno JOIN branch b ON b.bid
    = t.bid
    WHERE c.cname ILIKE p_customer_name;
END;
$$ LANGUAGE plpgsql;
```

Output:

Query 1: SELECT b.brname, SUM(l.lamtapproved) AS total_approved FROM branch b JOIN ternary t ON b.bid = t.bid JOIN loan_application l ON t.lno = l.lno WHERE l.ldate BETWEEN '2019-06-01' AND '2020-06-01' GROUP BY b.brname ORDER BY b.brname

brname	total_approved
Aundh	550000
J.M.Road	100000
M.G. ROAD	250000
Pimpri	800000

Query 2: SELECT DISTINCT c.* FROM customer c JOIN ternary t ON c.cno = t.cno JOIN branch b ON b.bid = t.bid WHERE b.brname = 'Aundh'

cno	cname	caddr	city
101	Asha	Karve Nagar	Pune
105	Anita	Aundh	Pune

Query 3: SELECT SUM(lamtrequired) AS total_required FROM loan_application

total_required
4995000

Query 4: SELECT * FROM branch

bid	brname	brcity
1	Aundh	Pune
2	Pimpri	Pune
3	Deccan	Pune
4	M.G. ROAD	Pune
5	J.M.Road	Pune

Query 5: SELECT * FROM fn_customer_loans('Asha')

loan_no	required_amount	approved_amount	loan_date	branch_name
1001	500000	450000	2019-06-10	Aundh

Assignment 8 - Bank Database

Same bank schema with insert, view and stored function questions.

Commands :

```
DROP TABLE IF EXISTS ternary, loan_application,
customer, branch CASCADE; CREATE TABLE branch (
    bid INT PRIMARY KEY,
    brname VARCHAR(30) NOT
    NULL UNIQUE, brcity
    VARCHAR(20) NOT NULL
);
CREATE TABLE
    customer ( cno
    INT PRIMARY
    KEY,
    cname VARCHAR(20)
    NOT NULL, caddr
    VARCHAR(35),
    city VARCHAR(20) NOT NULL
);
CREATE TABLE
    loan_application (
    lno INT PRIMARY
    KEY,
    lamtrequired NUMERIC(12,2) NOT NULL CHECK (lamtrequired > 0),
    lamtapproved NUMERIC(12,2) NOT NULL CHECK
    (lamtapproved >
    0)
    , ldate DATE NOT NULL
);
CREATE TABLE ternary (
    bid INT REFERENCES
    branch(bid), cno INT
    REFERENCES
    customer(cno),
    lno INT REFERENCES
    loan_application(lno), PRIMARY
    KEY (bid, cno, lno)
);
INSERT INTO branch VALUES
(1,'Aundh','Pune'),
(2,'Pimpri','Pune'),
(3,'Deccan','Pune'),
(4,'M.G. ROAD','Pune'),
(5,'J.M.Road','Pune');
INSERT INTO customer
VALUES (101,'Asha','Karve
Nagar','Pune'),
(102,'Amit','Baner','Pune'),
(103,'Pooja','Nigdi','Pune'),
(104,'Rohit','Camp','Pune'),
```

```

(105,'Anita','Aundh','Pune');
INSERT INTO
loan_application VALUES
(1001,500000,450000,'2019-
06-10'),
(1002,800000,800000,'2020-01-15'),
(1003,300000,250000,'2020-05-14'),
(1004,1200000,1000000,'2021-04-12'),
(1005,150000,100000,'2019-12-12');
INSERT INTO ternary VALUES
(1,101,1001),
(2,102,1002),
(2,103,1003),
(3,104,1004),
(2,105,1005);
SELECT l.*
FROM loan_application l
JOIN ternary t ON l.lno
= t.lno JOIN branch b
ON b.bid = t.bid
WHERE b.brname =
'Pimpri';
SELECT
cname
FROM
customer
WHERE cname ILIKE 'A%';
SELECT c.*
FROM customer c
JOIN ternary t ON c.cno = t.cno
JOIN loan_application l ON
l.lno = t.lno WHERE
l.lamtrequired = 800000;
CREATE OR REPLACE VIEW
vw_pimpri_customers AS SELECT
DISTINCT c.*
FROM customer c
JOIN ternary t ON
c.cno = t.cno JOIN
branch b ON b.bid =
t.bid WHERE b.brname
= 'Pimpri';
SELECT * FROM vw_pimpri_customers;
CREATE OR REPLACE FUNCTION
fn_branch_customer_count(p_branch_name VARCHAR) RETURNS
TEXT AS $$
DECLARE
    v_count INT;
BEGIN
    SELECT COUNT(DISTINCT t.cno)
    INTO
    v_count

```

```
FROM branch
b
JOIN ternary t ON b.bid = t.bid
WHERE b.brname ILIKE p_branch_name;
IF v_count = 0 THEN
    RETURN 'Invalid branch
name'; END IF;
RETURN 'Customer count for ' || p_branch_name || ' = ' || v_count;
END;
$$ LANGUAGE plpgsql;
SELECT
fn_branch_customer_count('Pimpri');
SELECT
fn_branch_customer_count('Unknown');
```

Output:

Query 1: SELECT I.* FROM loan_application I JOIN ternary t ON I.lno = t.lno JOIN branch b ON b.bid = t.bid WHERE b.brname = 'Pimpri'

Ino	lamtrequired	lamtapproved	ldate
1002	800000	800000	2020-01-15
1003	300000	250000	2020-05-14
1005	150000	100000	2019-12-12

Query 2: SELECT cname FROM customer WHERE cname ILIKE 'A%'

cname
Asha
Amit
Anita

Query 3: SELECT c.* FROM customer c JOIN ternary t ON c.cno = t.cno JOIN loan_application I ON I.lno = t.lno WHERE I.lamtrequired = 800000

cno	cname	caddr	city
102	Amit	Baner	Pune

Query 4: SELECT * FROM vw_pimpri_customers

cno	cname	caddr	city
102	Amit	Baner	Pune
103	Pooja	Nigdi	Pune
105	Anita	Aundh	Pune

Query 5: SELECT fn_branch_customer_count('Pimpri')

fn_branch_customer_count('Pimpri')
3

Query 6: SELECT fn_branch_customer_count('Unknown')

fn_branch_customer_count('Unknown')
0

Assignment 9 - Student Competition

Student and Competition with many-to-many relationship.

Commands :

```
DROP TABLE IF EXISTS student_competition, competition,
student CASCADE; CREATE TABLE student (
    sid INT PRIMARY KEY,
    sname VARCHAR(30)
    NOT NULL, class
    VARCHAR(10) NOT
    NULL
);
CREATE TABLE
    competition ( cno
    INT PRIMARY
    KEY,
    cname VARCHAR(30)
    NOT NULL, ctype
    VARCHAR(20) NOT
    NULL
);
CREATE TABLE
    student_competition ( sid
    INT REFERENCES
    student(sid),
    cno INT REFERENCES
    competition(cno), rank INT,
    year INT,
    PRIMARY KEY (sid, cno)
);
INSERT INTO student VALUES
(1,'Aarav','SYBCA'),
(2,'Pooja','SYBCA'),
(3,'Rohan','FYBCA'),
(4,'Aditi','TYBCA'),
(5,'Nikhil','SYBCA');
INSERT INTO competition
VALUES
(11,'Football','National'),
(12,'Quiz','State'),
(13,'Coding','National'),
(14,'Debate','University');
INSERT INTO
student_competition VALUES
(1,11,2,2018),
(2,11,1,2018),
(3,11,5,2019),
(4,11,2,2018),
(2,13,3,2019),
(4,13,1,2020),
(5,13,2,2021),
(3,12,4,2018),
```

```

(5,12,2,2018),
(2,14,2,2018);
SELECT sid,
sname FROM
student;
SELECT
sname FROM
student
ORDER BY
sname;
SELECT DISTINCT s.sid,
s.sname FROM student
s
JOIN student_competition sc ON
s.sid = sc.sid JOIN competition c
ON c.cno = sc.cno
WHERE c.ctype =

'National'; SELECT *

FROM competition;

CREATE OR REPLACE FUNCTION
trg_validate_student_id() RETURNS TRIGGER
AS $$
BEGIN
    IF NEW.sid < 0 THEN
        RAISE EXCEPTION 'Student id must be
greater than zero'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
before_insert_student BEFORE
INSERT ON student
FOR EACH ROW
EXECUTE FUNCTION trg_validate_student_id();
CREATE OR REPLACE VIEW vw_student_competition AS
SELECT s.sid, s.sname, s.class, c.cname, c.ctype,
sc.rank, sc.year FROM student s
JOIN student_competition sc ON s.sid = sc.sid
JOIN competition c ON c.cno =

sc.cno; SELECT * FROM

vw_student_competition;

```

Output:

Query 1: SELECT sid, sname FROM student

sid	sname
1	Aarav
2	Pooja
3	Rohan
4	Aditi
5	Nikhil

Query 2: SELECT sname FROM student ORDER BY sname

sname
Aarav
Aditi
Nikhil
Pooja
Rohan

Query 3: SELECT DISTINCT s.sid, s.sname FROM student s JOIN student_competition sc ON s.sid = sc.sid JOIN competition c ON c.cno = sc.cno WHERE c.ctype = 'National'

sid	sname
1	Aarav
2	Pooja
3	Rohan
4	Aditi
5	Nikhil

Query 4: SELECT * FROM competition

cno	cname	ctype
11	Football	National
12	Quiz	State
13	Coding	National
14	Debate	University

Query 5: SELECT * FROM vw_student_competition

sid	sname	class	cname	ctype	rank	year
1	Aarav	SYBCA	Football	National	2	2018
2	Pooja	SYBCA	Football	National	1	2018
3	Rohan	FYBCA	Football	National	5	2019
4	Aditi	TYBCA	Football	National	2	2018
2	Pooja	SYBCA	Coding	National	3	2019
4	Aditi	TYBCA	Coding	National	1	2020
5	Nikhil	SYBCA	Coding	National	2	2021
3	Rohan	FYBCA	Quiz	State	4	2018
5	Nikhil	SYBCA	Quiz	State	2	2018
2	Pooja	SYBCA	Debate	University	2	2018

Assignment 10 - Bus Transport System

Slip label is partly unclear in OCR; this is the bus-transport slip that comes before Slip 5.

Commands :

```
DROP TABLE IF EXISTS bus_driver, driver, bus,  
route CASCADE;
```

```
CREATE TABLE route (  
    route_no INT  
    PRIMARY KEY, source  
    VARCHAR(20),  
    destination  
    VARCHAR(20),  
    no_of_stations INT  
);  
CREATE TABLE bus (  
    bus_no INT PRIMARY  
    KEY, capacity INT  
    NOT NULL,  
    depot_name  
    VARCHAR(20),  
    route_no INT REFERENCES route(route_no)  
);  
CREATE TABLE driver  
    (  
    driver_no INT  
    PRIMARY KEY,  
    driver_name VARCHAR(20)  
    NOT NULL, license_no INT  
    UNIQUE NOT NULL, address  
    VARCHAR(20),  
    age INT,  
    salary NUMERIC(10,2)  
);  
CREATE TABLE bus_driver (  
    bus_no INT REFERENCES  
    bus(bus_no), driver_no INT  
    REFERENCES driver(driver_no),  
    shift INT CHECK (shift IN  
    (1,2)),  
    duty_date DATE,  
    PRIMARY KEY (bus_no, driver_no, duty_date)  
);  
INSERT INTO route VALUES  
(1,'Pune','Mumbai',18),  
(2,'Kothrud','Swargate',9),  
(3,'Nigdi','Hinjewadi',15),  
(4,'Swargate','Aundh',14);  
INSERT INTO bus VALUES  
(12,42,'Kothrud',4),  
(15,40,'Kothrud',4),  
(18,55,'Pune Station',1),  
(21,35,'Nigdi',3),
```

```

(25,40,'Kothrud',2);
INSERT INTO driver VALUES
(201,'Ramesh',9001,'Pune',45,320
00),
(202,'Sunil',9002,'Satara',39,28000),
(203,'Mahesh',9003,'Pune',52,35000),
(204,'Anil',9004,'Kolhapur',29,26000),
(205,'Kiran',9005,'Pune',41,31000);
INSERT INTO
bus_driver VALUES
(12,201,1,'2013-09-
26'),
(12,202,2,'2013-09-26'),
(15,203,1,'2013-09-26'),
(18,205,1,'2013-10-01'),
(25,201,2,'2013-10-03'),
(18,203,1,'2013-10-05');
SELECT *
FROM
driver
WHERE age
> 40;
SELECT
driver_name
FROM driver
WHERE salary > 30000;
INSERT INTO route VALUES
(5,'Pune','Nashik',16),
(6,'Pimpri','Shivajinagar',11);
SELECT
d.driver_name
FROM driver d
JOIN bus_driver bd ON d.driver_no = bd.driver_no
WHERE bd.bus_no = 12;
CREATE OR REPLACE FUNCTION
trg_driver_age_check() RETURNS TRIGGER
AS $$
BEGIN
    IF NEW.age BETWEEN 18 AND 25 THEN
        RAISE EXCEPTION
        'Invalid input'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
before_insert_driver BEFORE
INSERT ON driver
FOR EACH ROW
EXECUTE FUNCTION trg_driver_age_check();
CREATE OR REPLACE FUNCTION fn_buses_on_route(p_route_no INT)
RETURNS TABLE (
    bus_no INT,

```

```
        capacity INT,  
        depot_name VARCHAR(20)  
    ) AS $$  
    BEGIN  
        RETURN QUERY  
        SELECT b.bus_no, b.capacity,  
               b.depot_name FROM bus b  
        WHERE b.route_no = p_route_no;  
    END;  
    $$ LANGUAGE plpgsql;  
    SELECT * FROM fn_buses_on_route(4);
```

Output:

Query 1: SELECT * FROM driver WHERE age > 40

driver_no	driver_name	license_no	address	age	salary
201	Ramesh	9001	Pune	45	32000
203	Mahesh	9003	Pune	52	35000
205	Kiran	9005	Pune	41	31000

Query 2: SELECT driver_name FROM driver WHERE salary > 30000

driver_name
Ramesh
Mahesh
Kiran

Query 3: SELECT d.driver_name FROM driver d JOIN bus_driver bd ON d.driver_no = bd.driver_no WHERE bd.bus_no = 12

driver_name
Ramesh
Sunil

Query 4: SELECT * FROM fn_buses_on_route(4)

bus_no	capacity	depot_name
12	42	Kothrud
15	40	Kothrud

Assignment 11 - Bus Transport System

Bus, Route, Driver with view and cursor function.

Commands :

```
DROP TABLE IF EXISTS bus_driver, driver, bus,
```

```
route CASCADE; CREATE TABLE route (
```

```
    route_no INT  
    PRIMARY KEY, source  
    VARCHAR(20),  
    destination  
    VARCHAR(20),  
    no_of_stations INT
```

```
);
```

```
CREATE TABLE bus (
```

```
    bus_no INT PRIMARY  
    KEY, capacity INT  
    NOT NULL,  
    depot_name  
    VARCHAR(20),  
    route_no INT REFERENCES route(route_no)
```

```
);
```

```
CREATE TABLE driver
```

```
( driver_no INT  
    PRIMARY KEY,  
    driver_name VARCHAR(20)  
    NOT NULL, license_no INT  
    UNIQUE NOT NULL, address  
    VARCHAR(20),  
    age INT,  
    salary NUMERIC(10,2)
```

```
);
```

```
CREATE TABLE bus_driver (
```

```
    bus_no INT REFERENCES  
    bus(bus_no), driver_no INT  
    REFERENCES driver(driver_no),  
    shift INT CHECK (shift IN  
    (1,2)),  
    duty_date DATE,  
    PRIMARY KEY (bus_no, driver_no, duty_date)
```

```
);
```

```
INSERT INTO route VALUES
```

```
(1,'Pune','Mumbai',18),  
(2,'Kothrud','Swargate',9),  
(3,'Nigdi','Hinjewadi',15),  
(4,'Swargate','Aundh',14);
```

```
INSERT INTO bus VALUES
```

```
(12,42,'Kothrud',4),  
15,40,'Kothrud',4),  
(18,55,'Pune Station',1),  
(21,35,'Nigdi',3),  
(25,40,'Kothrud',2);
```

```
INSERT INTO driver VALUES
```



```

(201,'Ramesh',9001,'Pune',45,320
00),
(202,'Sunil',9002,'Satara',39,28000),
(203,'Mahesh',9003,'Pune',52,35000),
(204,'Anil',9004,'Kolhapur',29,26000),
(205,'Kiran',9005,'Pune',41,31000);
INSERT INTO
bus_driver VALUES
(12,201,1,'2013-09-
26'),
(12,202,2,'2013-09-26'),
(15,203,1,'2013-09-26'),
(18,205,1,'2013-10-01'),
(25,201,2,'2013-10-03'),
(18,203,1,'2013-10-05');
SELECT
driver_name
FROM driver;
SELECT DISTINCT r.*
FROM route r
JOIN bus b ON r.route_no =
b.route_no WHERE b.capacity
= 40;
INSERT INTO route VALUES
(5,'Pune','Nashik',16),
(6,'Pimpri','Shivajinagar',11);
SELECT *
FROM bus
WHERE route_no = 4;
CREATE OR REPLACE VIEW
vw_driver_duty_26sep2013 AS SELECT d.*,
bd.bus_no, bd.shift, bd.duty_date
FROM driver d
JOIN bus_driver bd ON d.driver_no =
bd.driver_no WHERE bd.duty_date =
'2013-09-26';
SELECT * FROM vw_driver_duty_26sep2013;
CREATE OR REPLACE FUNCTION
fn_driver_duty_dates(p_driver_name VARCHAR) RETURNS
TABLE (duty_date DATE) AS $$
DECLARE
    rec
    RECORD;
    cur
    CURSOR
    FOR
        SELECT bd.duty_date
        FROM bus_driver bd
        JOIN driver d ON d.driver_no =
        bd.driver_no WHERE
        d.driver_name ILIKE
        p_driver_name ORDER BY
        bd.duty_date;

```

```
BEGIN
  OPEN cur;
  LOOP
    FETCH cur INTO rec;
    EXIT WHEN NOT
    FOUND;
    duty_date : rec.duty_date;
    RETURN
  NEXT; END
  LOOP;
  CLOSE cur;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM fn_driver_duty_dates('Ramesh');
```

Output:

Query 1: SELECT driver_name FROM driver

driver_name
Ramesh
Sunil
Mahesh
Anil
Kiran

Query 2: SELECT DISTINCT r.* FROM route r JOIN bus b ON r.route_no = b.route_no WHERE b.capacity = 40

route_no	source	destination	no_of_stations
4	Swargate	Aundh	14
2	Kothrud	Swargate	9

Query 3: SELECT * FROM bus WHERE route_no = 4

bus_no	capacity	depot_name	route_no
12	42	Kothrud	4
15	40	Kothrud	4

Query 4: SELECT * FROM vw_driver_duty_26sep2013

driver_no	driver_name	license_no	address	age	salary	bus_no	shift	duty_date
201	Ramesh	9001	Pune	45	32000	12	1	2013-09-26
202	Sunil	9002	Satara	39	28000	12	2	2013-09-26
203	Mahesh	9003	Pune	52	35000	15	1	2013-09-26

Query 5: SELECT * FROM fn_driver_duty_dates('Ramesh')

duty_date
2013-09-26
2013-10-03

Assignment 12 - Student Competition

Student competition slip with competition-wise count, view and function.

Commands :

```
DROP TABLE IF EXISTS student_competition, competition,
student CASCADE; CREATE TABLE student (
    sid INT PRIMARY KEY,
    sname VARCHAR(30)
    NOT NULL, class
    VARCHAR(10) NOT
    NULL
);
CREATE TABLE
    competition ( cno
    INT PRIMARY
    KEY,
    cname VARCHAR(30)
    NOT NULL, ctype
    VARCHAR(20) NOT
    NULL
);
CREATE TABLE
    student_competition ( sid
    INT REFERENCES
    student(sid),
    cno INT REFERENCES
    competition(cno), rank INT,
    year INT,
    PRIMARY KEY (sid, cno)
);
INSERT INTO student VALUES
(1,'Aarav','SYBCA'),
(2,'Pooja','SYBCA'),
(3,'Rohan','FYBCA'),
(4,'Aditi','TYBCA'),
(5,'Nikhil','SYBCA');
INSERT INTO competition
VALUES
(11,'Football','National'),
(12,'Quiz','State'),
(13,'Coding','National'),
(14,'Debate','University');
INSERT INTO
student_competition VALUES
(1,11,2,2018),
(2,11,1,2018),
(3,11,5,2019),
(4,11,2,2018),
(2,13,3,2019),
(4,13,1,2020),
(5,13,2,2021),
(3,12,4,2018),
```

```

(5,12,2,2018),
(2,14,2,2018);
SELECT
sname
FROM
student;
SELECT DISTINCT s.sname
FROM student s
JOIN student_competition sc ON
s.sid = sc.sid JOIN competition c
ON c.cno = sc.cno
WHERE c.cname = 'Football';
SELECT s.*, sc.rank,
sc.year FROM student s
JOIN student_competition sc ON
s.sid = sc.sid WHERE sc.rank = 1;
SELECT c.cname, COUNT(sc.sid) AS
student_count FROM competition c
JOIN student_competition sc ON
c.cno = sc.cno GROUP BY c.cname
ORDER BY c.cname;
CREATE OR REPLACE VIEW
vw_students_year_2018 AS SELECT
DISTINCT s.sid, s.sname, s.class
FROM student s
JOIN student_competition sc ON
s.sid = sc.sid WHERE sc.year =
2018;
SELECT * FROM vw_students_year_2018;
CREATE OR REPLACE FUNCTION
fn_competitions_of_student(p_student_name VARCHAR) RETURNS
TABLE (
    competition_name
    VARCHAR(30),
    competition_type
    VARCHAR(20),    rank
    INT,
    participation_year INT
) AS
$$
BEGIN
    RETURN QUERY
    SELECT c.cname, c.ctype,
    sc.rank, sc.year FROM student s
    JOIN student_competition sc ON
    s.sid = sc.sid JOIN competition c
    ON c.cno = sc.cno
    WHERE s.sname ILIKE p_student_name;
END;
$$ LANGUAGE plpgsql;
SELECT * FROM fn_competitions_of_student('Pooja');

```

Output:**Query 1: SELECT sname FROM student**

sname
Aarav
Pooja
Rohan
Aditi
Nikhil

Query 2: SELECT DISTINCT s.sname FROM student s JOIN student_competition sc ON s.sid = sc.sid JOIN competition c ON c.cno = sc.cno WHERE c.cname = 'Football'

sname
Aarav
Pooja
Rohan
Aditi

Query 3: SELECT s.*, sc.rank, sc.year FROM student s JOIN student_competition sc ON s.sid = sc.sid WHERE sc.rank = 1

sid	sname	class	rank	year
2	Pooja	SYBCA	1	2018
4	Aditi	TYBCA	1	2020

Query 4: SELECT c.cname, COUNT(sc.sid) AS student_count FROM competition c JOIN student_competition sc ON c.cno = sc.cno GROUP BY c.cname ORDER BY c.cname

cname	student_count
Coding	3
Debate	1
Football	4
Quiz	2

Query 5: SELECT * FROM vw_students_year_2018

sid	sname	class
1	Aarav	SYBCA
2	Pooja	SYBCA
4	Aditi	TYBCA
3	Rohan	FYBCA
5	Nikhil	SYBCA

Query 6: SELECT * FROM fn_competitions_of_student('Pooja')

competition_name	competition_type	rank	participation_year
Football	National	1	2018
Coding	National	3	2019
Debate	University	2	2018

Assignment 13- Railway Reservation System

Train, Passenger and Ticket with trigger and cursor function.

Commands :

```
DROP TABLE IF EXISTS ticket,  
passenger, train CASCADE; CREATE
```

```
TABLE train (  
    train_no INT PRIMARY  
    KEY, train_name  
    VARCHAR(30) NOT NULL,  
    depart_time TIME,  
    arrival_time TIME,  
    source_stn  
    VARCHAR(20),  
    dest_stn  
    VARCHAR(20),  
    no_of_res_bogies  
    INT, bogie_capacity  
    INT  
);  
CREATE TABLE passenger (  
    passenger_id INT  
    PRIMARY KEY,  
    passenger_name VARCHAR(20)  
    NOT NULL, address  
    VARCHAR(30),  
    age INT,  
    gender CHAR(1)  
);  
CREATE TABLE ticket (  
    ticket_no INT  
    PRIMARY KEY,  
    train_no INT REFERENCES  
    train(train_no), passenger_id INT  
    REFERENCES passenger(passenger_id),  
    bogie_no INT,  
    no_of_berths  
    INT, tdate  
    DATE,  
    ticket_amt NUMERIC(10,2),  
    ticket_status CHAR(1) CHECK (ticket_status IN ('W','C'))  
);  
INSERT INTO train VALUES  
(101,'Shatabdi Express','06:00','10:00','Pune','Mumbai',12,30),  
(102,'Koyana Express','07:30','14:00','Pune','Kolhapur',8,30),  
(103,'Rajdhani Express','20:00','06:00','Mumbai','Delhi',15,30),  
(104,'Deccan Queen','17:00','20:30','Pune','Mumbai',11,30);  
INSERT INTO passenger  
VALUES  
(1,'Mr.Raj','Pune',32,'M'),
```

```

(2,'Riya','Mumbai',4,'F'),
(3,'Suresh','Satara',65,'M'),
(4,'Anita','Pune',28,'F'),
(5,'Mahesh','Delhi',62,'M'),
(6,'Aakash','Nagpur',34,'M'),
(7,'Meena','Mumbai',31,'F');
INSERT INTO ticket VALUES
(5001,101,1,1,2,'2021-04-
12',1200.00,'C'),
(5002,102,1,2,1,'2021-04-12',900.00,'W'),
(5003,101,2,3,1,'2021-04-12',0.00,'C'),
(5004,103,3,1,2,'2021-04-12',4500.00,'C'),
(5005,103,4,2,1,'2021-04-12',4300.00,'W'),
(5006,102,5,1,2,'2019-12-12',1100.00,'C'),
(5007,101,3,4,1,'2019-12-12',1200.00,'C'),
(5008,103,5,3,1,'2019-12-12',4600.00,'W');
SELECT
p.passenger_name
FROM passenger p
JOIN ticket t ON p.passenger_id = t.passenger_id
JOIN train tr ON tr.train_no =
t.train_no WHERE tr.train_name =
'Shatabdi Express';
INSERT INTO ticket VALUES
(5009,101,4,5,1,'2021-04-
12',1200.00,'C'),
(5010,103,6,4,1,'2021-04-12',4300.00,'C');

SELECT * FROM

train; SELECT

t.*
FROM ticket t
JOIN passenger p ON p.passenger_id =
t.passenger_id WHERE p.passenger_name
= 'Mr.Raj';
CREATE OR REPLACE FUNCTION
trg_passenger_age_notice() RETURNS
TRIGGER AS $$
BEGIN
    IF NEW.age > 5 THEN
        RAISE NOTICE 'Age above 5 will be
        charged full fare'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
after_insert_passenger AFTER
INSERT ON passenger
FOR EACH ROW
EXECUTE FUNCTION trg_passenger_age_notice();
CREATE OR REPLACE FUNCTION

```



```

fn_total_berths_on_date(p_date DATE) RETURNS INT
AS $$
DECLARE
    cur CURSOR FOR
        SELECT
            no_of_berths
        FROM ticket
        WHERE tdate = p_date;
    v_berths INT;
    v_total INT : 0;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_berths;
        EXIT WHEN NOT FOUND;
        v_total : v_total +
    v_berths; END LOOP;
    CLOSE cur;
    RETURN v_total;
END;
$$ LANGUAGE plpgsql;

SELECT  fn total berths on date('2021-04-12');

```

Output:

Query 1: SELECT p.passenger_name FROM passenger p JOIN ticket t ON p.passenger_id = t.passenger_id JOIN train tr ON tr.train_no = t.train_no WHERE tr.train_name = 'Shatabdi Express'

passenger_name
Mr.Raj
Riya
Suresh

Query 2: SELECT * FROM train

train_no	train_name	depart_time	arrival_time	source_stn	dest_stn	no_of_res_bogies	bogie_capacity
101	Shatabdi Express	06:00	10:00	Pune	Mumbai	12	30
102	Koyana Express	07:30	14:00	Pune	Kolhapur	8	30
103	Rajdhani Express	20:00	06:00	Mumbai	Delhi	15	30
104	Deccan Queen	17:00	20:30	Pune	Mumbai	11	30

Query 3: SELECT t.* FROM ticket t JOIN passenger p ON p.passenger_id = t.passenger_id WHERE p.passenger_name = 'Mr.Raj'

ticket_no	train_no	passenger_id	bogie_no	no_of_berths	tdate	ticket_amt	ticket_status
5001	101	1	1	2	2021-04-12	1200	C
5002	102	1	2	1	2021-04-12	900	W

Query 4: SELECT fn_total_berths_on_date('2021-04-12')

fn_total_berths_on_date('2021-04-12')
2

Assignment 14- Railway Reservation System

Railway slip with confirmed passenger view and waiting booking function.

Commands :

```
DROP TABLE IF EXISTS ticket,
passenger, train CASCADE; CREATE
TABLE train (
    train_no INT PRIMARY
    KEY, train_name
    VARCHAR(30) NOT NULL,
    depart_time TIME,
    arrival_time TIME,
    source_stn
    VARCHAR(20),
    dest_stn
    VARCHAR(20),
    no_of_res_bogies
    INT, bogie_capacity
    INT
);
CREATE TABLE passenger (
    passenger_id INT
    PRIMARY KEY,
    passenger_name VARCHAR(20)
    NOT NULL, address
    VARCHAR(30),
    age INT,
    gender CHAR(1)
);
CREATE TABLE ticket (
    ticket_no INT
    PRIMARY KEY,
    train_no INT REFERENCES
    train(train_no), passenger_id INT
    REFERENCES passenger(passenger_id),
    bogie_no INT,
    no_of_berths
    INT, tdate
    DATE,
    ticket_amt NUMERIC(10,2),
    ticket_status CHAR(1) CHECK (ticket_status IN ('W','C'))
);
INSERT INTO train VALUES
(101,'Shatabdi Express','06:00','10:00','Pune','Mumbai',12,30),
(102,'Koyana Express','07:30','14:00','Pune','Kolhapur',8,30),
(103,'Rajdhani Express','20:00','06:00','Mumbai','Delhi',15,30);
INSERT INTO passenger
VALUES
(1,'Mr.Raj','Pune',32,'M'),
(2,'Riya','Mumbai',4,'F'),
(3,'Suresh','Satara',65,'M'),
```

```

(4,'Anita','Pune',28,'F'),
(5,'Mahesh','Delhi',62,'M'),
(6,'Aakash','Nagpur',34,'M'),
(7,'Meena','Mumbai',31,'F');
INSERT INTO ticket VALUES
(5001,101,1,1,2,'2021-04-
12',1200.00,'C'),
(5002,102,1,2,1,'2021-04-12',900.00,'W'),
(5003,101,2,3,1,'2021-04-12',0.00,'C'),
(5004,103,3,1,2,'2021-04-12',4500.00,'C'),
(5005,103,4,2,1,'2021-04-12',4300.00,'W'),
(5006,102,5,1,2,'2019-12-12',1100.00,'C'),
(5007,103,6,4,1,'2021-04-12',4300.00,'C'),
(5008,103,7,5,1,'2021-04-12',4300.00,'C');
SELECT
p.passenger_name
FROM passenger p
JOIN ticket t ON p.passenger_id =
t.passenger_id JOIN train tr ON
tr.train_no = t.train_no
WHERE tr.train_name = 'Koyana Express';
SELECT COUNT(*) AS
confirmed_count FROM ticket
t
JOIN train tr ON tr.train_no = t.train_no
WHERE tr.train_name =
'Rajdhani Express' AND
t.ticket_status = 'C';
SELECT *
FROM
passenger
ORDER BY
age;
SELECT
passenger_name
FROM passenger
WHERE age < 5;
CREATE OR REPLACE VIEW
vw_confirmed_passengers AS SELECT p.*,
t.ticket_no, t.train_no, t.tdate
FROM passenger p
JOIN ticket t ON p.passenger_id =
t.passenger_id WHERE t.ticket_status
= 'C';
SELECT * FROM vw_confirmed_passengers;
CREATE OR REPLACE FUNCTION
fn_waiting_bookings_on_date(p_date DATE) RETURNS
TABLE (
train_name
VARCHAR(30),
waiting_bookings
BIGINT

```

```
) AS
$$
BEGIN
N
    RETURN QUERY
    SELECT tr.train_name,
    COUNT(*) FROM train tr
    JOIN ticket t ON tr.train_no = t.train_no
    WHERE t.tdate = p_date
    AND t.ticket_status
    = 'W' GROUP BY
    tr.train_name
    ORDER BY tr.train_name;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM fn_waiting_bookings_on_date('2021-04-12');
```

Output:

Query 1: SELECT p.passenger_name FROM passenger p JOIN ticket t ON p.passenger_id = t.passenger_id JOIN train tr ON tr.train_no = t.train_no WHERE tr.train_name = 'Koyana Express'

passenger_name
Mr.Raj
Mahesh

Query 2: SELECT COUNT(*) AS confirmed_count FROM ticket t JOIN train tr ON tr.train_no = t.train_no WHERE tr.train_name = 'Rajdhani Express' AND t.ticket_status = 'C'

confirmed_count
3

Query 3: SELECT * FROM passenger ORDER BY age

passenger_id	passenger_name	address	age	gender
2	Riya	Mumbai	4	F
4	Anita	Pune	28	F
7	Meena	Mumbai	31	F
1	Mr.Raj	Pune	32	M
6	Aakash	Nagpur	34	M
5	Mahesh	Delhi	62	M
3	Suresh	Satara	65	M

Query 4: SELECT passenger_name FROM passenger WHERE age < 5

passenger_name
Riya

Query 5: SELECT * FROM vw_confirmed_passengers

passenger_id	passenger_name	address	age	gender	ticket_no	train_no	tdate
1	Mr.Raj	Pune	32	M	5001	101	2021-04-12
2	Riya	Mumbai	4	F	5003	101	2021-04-12
3	Suresh	Satara	65	M	5004	103	2021-04-12
5	Mahesh	Delhi	62	M	5006	102	2019-12-12
6	Aakash	Nagpur	34	M	5007	103	2021-04-12
7	Meena	Mumbai	31	F	5008	103	2021-04-12

Query 6: SELECT * FROM fn_waiting_bookings_on_date('2021-04-12')

train_name	waiting_bookings
Koyana Express	1
Rajdhani Express	1

Assignment 15- Project Employee Database

Projects assigned to employees through many-to-many relationship.

Commands :

```
DROP TABLE IF EXISTS project_employee, employee,
```

```
project CASCADE; CREATE TABLE project (  
    pno INT PRIMARY KEY,  
    pname VARCHAR(30)  
    NOT NULL, ptype  
    VARCHAR(20),  
    duration INT CHECK (duration > 0)  
);
```

```
CREATE TABLE  
    employee ( eno  
    INT PRIMARY  
    KEY,  
    ename VARCHAR(20) NOT NULL,  
    qualification VARCHAR(15),  
    joining_date DATE  
);
```

```
CREATE TABLE project_employee (  
    pno INT REFERENCES  
    project(pno), eno INT  
    REFERENCES employee(en),  
    start_date DATE,  
    hours_worked  
    INT, PRIMARY KEY  
    (pno, eno)  
);
```

```
INSERT INTO project VALUES  
(1,'AI','Research',6),  
(2,'Cloud','Development',4),  
(3,'ERP','Support',2),  
(4,'Analytics','BI',5);  
INSERT INTO employee  
VALUES  
(1,'Rahul','M.Sc','2019-  
06-10'),  
(2,'Nehal','BCA','2020-01-01'),  
(3,'Sonal','M.Sc','2018-03-12'),  
(4,'Kapil','MCA','2017-07-20'),  
(5,'Vishal','B.Sc','2021-02-15');
```

```
INSERT INTO  
project_employee VALUES  
(1,1,'2017-09-20',120),  
(1,2,'2017-09-20',90),  
(2,3,'2018-01-10',110),  
(3,4,'2017-09-20',60),  
(4,5,'2020-04-18',100),  
(2,5,'2019-09-20',70),  
(4,2,'2021-03-01',80);  
SELECT p.pname, p.ptype,
```

```

pe.start_date FROM project p
JOIN project_employee pe ON p.pno
= pe.pno ORDER BY p.pname,
pe.start_date;
SELECT e.*
FROM employee e
JOIN project_employee pe ON
e.eno = pe.eno JOIN project p
ON p.pno = pe.pno
WHERE p.pname = 'AI';
SELECT
ename
FROM
employee
WHERE qualification = 'M.Sc';
SELECT COUNT(*) AS
employee_count FROM
employee;
CREATE OR REPLACE FUNCTION
trg_project_duration_check() RETURNS
TRIGGER AS $$
BEGIN
    IF NEW.duration < 0 THEN
        RAISE EXCEPTION 'Project duration must be
greater than zero'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
before_insert_project BEFORE
INSERT ON project
FOR EACH ROW
EXECUTE FUNCTION trg_project_duration_check();
CREATE OR REPLACE FUNCTION
fn_employees_of_project(p_project_name VARCHAR) RETURNS
TABLE (employee_name VARCHAR(20)) AS $$
BEGIN
    RETURN QUERY
    SELECT
    e.ename
    FROM
    employee e
    JOIN project_employee pe ON e.eno = pe.eno
    JOIN project p ON p.pno =
    pe.pno WHERE p.pname
    ILIKE p_project_name
    ORDER BY e.ename;
END;
$$ LANGUAGE plpgsql;
SELECT * FROM fn_employees_of_project('AI');

```


Output:

Query 1: SELECT p.pname, p.ptype, pe.start_date FROM project p JOIN project_employee pe ON p.pno = pe.pno ORDER BY p.pname, pe.start_date

pname	ptype	start_date
AI	Research	2017-09-20
AI	Research	2017-09-20
Analytics	BI	2020-04-18
Analytics	BI	2021-03-01
Cloud	Development	2018-01-10
Cloud	Development	2019-09-20
ERP	Support	2017-09-20

Query 2: SELECT e.* FROM employee e JOIN project_employee pe ON e.eno = pe.eno JOIN project p ON p.pno = pe.pno WHERE p.pname = 'AI'

eno	ename	qualification	joining_date
1	Rahul	M.Sc	2019-06-10
2	Nehal	BCA	2020-01-01

Query 3: SELECT ename FROM employee WHERE qualification = 'M.Sc'

ename
Rahul
Sonal

Query 4: SELECT COUNT(*) AS employee_count FROM employee

employee_count
5

Query 5: SELECT * FROM fn_employees_of_project('AI')

employee_name
Nehal
Rahul

Assignment 16- Project Employee Database

Project employee slip with delete trigger and descending employee view.

Commands :

```
DROP TABLE IF EXISTS project_employee, employee,
```

```
project CASCADE; CREATE TABLE project (  
    pno INT PRIMARY KEY,  
    pname VARCHAR(30)  
    NOT NULL, ptype  
    VARCHAR(20),  
    duration INT CHECK (duration > 0)  
);
```

```
CREATE TABLE  
    employee ( eno  
    INT PRIMARY  
    KEY,  
    ename VARCHAR(20) NOT NULL,  
    qualification VARCHAR(15),  
    joining_date DATE  
);
```

```
CREATE TABLE project_employee (  
    pno INT REFERENCES  
    project(pno), eno INT  
    REFERENCES employee(en),  
    start_date DATE,  
    hours_worked  
    INT, PRIMARY KEY  
    (pno, eno)  
);
```

```
INSERT INTO project VALUES  
(1,'AI','Research',6),  
(2,'Cloud','Development',4),  
(3,'ERP','Support',2),  
(4,'Analytics','BI',5);  
INSERT INTO employee  
VALUES  
(1,'Rahul','M.Sc','2019-  
06-10'),  
(2,'Nehal','BCA','2020-01-01'),  
(3,'Sonal','M.Sc','2018-03-12'),  
(4,'Kapil','MCA','2017-07-20'),  
(5,'Vishal','B.Sc','2021-02-15');
```

```
INSERT INTO  
project_employee VALUES  
(1,1,'2017-09-20',120),  
(1,2,'2017-09-20',90),  
(2,3,'2018-01-10',110),  
(3,4,'2017-09-20',60),  
(4,5,'2020-04-18',100),  
(2,5,'2019-09-20',70),  
(4,2,'2021-03-01',80);  
SELECT
```

```
ename
FROM
employee
WHERE ename ILIKE '%l';
INSERT INTO employee
VALUES
(6,'Minal','MBA','2022-
01-12'),
(7,'Pranil','MCA','2023-02-16');
SELECT DISTINCT p.*
FROM project p
JOIN project_employee pe ON p.pno
= pe.pno WHERE pe.start_date =
'2017-09-20';
SELECT *
FROM project
WHERE duration > 3;
CREATE OR REPLACE FUNCTION
trg_employee_delete_notice() RETURNS TRIGGER
AS $$
BEGIN
    RAISE NOTICE 'Employee record is
being deleted'; RETURN OLD;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
before_delete_employee BEFORE
DELETE ON employee
FOR EACH ROW
EXECUTE FUNCTION trg_employee_delete_notice();
CREATE OR REPLACE VIEW
vw_employee_desc AS SELECT *
FROM employee
ORDER BY ename DESC;
SELECT * FROM vw_employee_desc;
```

Output:

Query 1: SELECT ename FROM employee WHERE ename ILIKE '%l'

ename
Rahul
Nehal
Sonal
Kapil
Vishal

Query 2: SELECT DISTINCT p.* FROM project p JOIN project_employee pe ON p.pno = pe.pno WHERE pe.start_date = '2017-09-20'

pno	pname	ptype	duration
1	AI	Research	6
3	ERP	Support	2

Query 3: SELECT * FROM project WHERE duration > 3

pno	pname	ptype	duration
1	AI	Research	6
2	Cloud	Development	4
4	Analytics	BI	5

Query 4: SELECT * FROM vw_employee_desc

eno	ename	qualification	joining_date
5	Vishal	B.Sc	2021-02-15
3	Sonal	M.Sc	2018-03-12
1	Rahul	M.Sc	2019-06-10
7	Pranil	MCA	2023-02-16
2	Nehal	BCA	2020-01-01
6	Minal	MBA	2022-01-12
4	Kapil	MCA	2017-07-20

Assignment 17- Student Course Enrollment

Student, Course and many-to-many enrollment.

Commands :

```
DROP TABLE IF EXISTS enrollment, course,
student CASCADE; CREATE TABLE student (
    rollno INT PRIMARY
    KEY, sname
    VARCHAR(30) NOT
    NULL,
    phone VARCHAR(15)
);
CREATE TABLE course (
    ccode VARCHAR(10)
    PRIMARY KEY, cname
    VARCHAR(30) NOT NULL,
    duration VARCHAR(20) NOT NULL
);
CREATE TABLE enrollment (
    rollno INT REFERENCES
    student(rollno), ccode
    VARCHAR(10) REFERENCES
    course(ccode), PRIMARY KEY
    (rollno, ccode)
);
INSERT INTO student VALUES
(1,'Rohan','987650001'),
(2,'Riya','987650002'),
(3,'Amit','987650003'),
(4,'Pooja','987650004');
INSERT INTO course VALUES
('C101','PostgreSQL','6 months'),
('C102','Python','6 months'),
('C103','Java','12 months');
INSERT INTO enrollment
VALUES (1,'C101'),
(1,'C102'),
(2,'C101'),
(3,'C103'),
(4,'C101');
SELECT *
FROM course
WHERE duration = '6 months';
SELECT s.rollno, s.sname, c.ccode,
c.cname, c.duration FROM student s
JOIN enrollment e ON s.rollno = e.rollno
JOIN course c ON c.ccode =
e.ccode ORDER BY s.sname;
DELETE FROM
student WHERE
sname = 'Rohan';
UPDATE student
```

```

SET phone = '987654321'
WHERE sname = 'Riya';
CREATE OR REPLACE FUNCTION
trg_validate_student_basic() RETURNS
TRIGGER AS $$
BEGIN
    IF NEW.rollno < 0 OR NEW.sname IS NULL THEN
        RAISE EXCEPTION 'Roll number must be positive and
        student name cannot be null'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
before_insert_student_basic BEFORE
INSERT ON student
FOR EACH ROW
EXECUTE FUNCTION trg_validate_student_basic();
CREATE OR REPLACE FUNCTION
fn_course_details(p_ccode VARCHAR) RETURNS TABLE
(
    course_name VARCHAR(30),
    duration VARCHAR(20)
) AS
$$
BEGIN
    RETURN QUERY
    SELECT cname,
    course.duration FROM
    course
    WHERE ccode = p_ccode;
END;
$$ LANGUAGE plpgsql;
SELECT * FROM fn_course_details('C101');

```

Output:

Query 1: SELECT * FROM course WHERE duration = '6 months'

ccode	cname	duration
C101	PostgreSQL	6 months
C102	Python	6 months

Query 2: SELECT s.rollno, s.sname, c.ccode, c.cname, c.duration FROM student s JOIN enrollment e ON s.rollno = e.rollno JOIN course c ON c.ccode = e.ccode ORDER BY s.sname

rollno	sname	ccode	cname	duration
3	Amit	C103	Java	12 months
4	Pooja	C101	PostgreSQL	6 months
2	Riya	C101	PostgreSQL	6 months
1	Rohan	C101	PostgreSQL	6 months
1	Rohan	C102	Python	6 months

Query 3: DELETE FROM student WHERE sname = 'Rohan'

ERROR: Foreign key constraint failed because related child rows still exist.

Query 4: SELECT * FROM fn_course_details('C101')

course_name	duration
PostgreSQL	6 months

Assignment 18- Employee Investor

Employee and Investor with a practical validation trigger.

Commands :

```
DROP TABLE IF EXISTS investor,  
employee CASCADE; CREATE TABLE  
employee (  
    emp_id INT PRIMARY  
    KEY, emp_name  
    VARCHAR(30) NOT NULL,  
    address  
    VARCHAR(30),  
    bdate DATE  
);  
CREATE TABLE  
    investor ( inv_no  
    INT PRIMARY KEY,  
    inv_name VARCHAR(30) NOT NULL,  
    inv_date DATE,  
    inv_amt  
    NUMERIC(12,2)  
);  
INSERT INTO employee  
VALUES  
(1,'Mohan','Pune','1990-  
01-10'),  
(2,'Rakesh','Mumbai','1989-09-12'),  
(3,'Mehul','Nashik','1992-07-08'),  
(4,'Sneha','Pune','1995-02-17');  
INSERT INTO investor  
VALUES  
(101,'Mahesh','2024-01-  
15',20000),  
(102,'Mira','2024-02-01',18000),  
(103,'Rohit','2024-03-05',25000),  
(104,'Mohan','2024-03-20',15000);  
SELECT  
    emp_name  
FROM  
    employee;  
SELECT *  
FROM investor  
WHERE inv_name ILIKE 'M%';  
UPDATE investor  
SET inv_amt = inv_amt * 1.10;  
DELETE FROM  
    investor WHERE  
    inv_amt = 20000;  
CREATE OR REPLACE FUNCTION  
trg_validate_investment_amount() RETURNS  
TRIGGER AS $$
```



```

BEGIN
    IF NEW.inv_amt > 2000 THEN
        RAISE EXCEPTION 'Investment amount must be less
        than or equal to 2000'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
before_insert_investor BEFORE
INSERT ON investor
FOR EACH ROW
EXECUTE FUNCTION trg_validate_investment_amount();
CREATE OR REPLACE FUNCTION
fn_investor_count(p_investor_name VARCHAR) RETURNS TEXT
AS $$
DECLARE
    v_count INT;
BEGIN
    SELECT COUNT(*)
    INTO
    v_count
    FROM
    investor
    WHERE inv_name ILIKE p_investor_name;
    IF v_count = 0 THEN
        RETURN 'Invalid
        investor name'; END IF;
    RETURN 'Investor count for ' | p_investor_name | ' = ' | v_count;
END;
$$ LANGUAGE plpgsql;
SELECT
fn_investor_count('Mohan');
SELECT
fn_investor_count('Unknown');

```

Output:

Query 1: SELECT emp_name FROM employee

emp_name
Mohan
Rakesh
Mehul
Sneha

Query 2: SELECT * FROM investor WHERE inv_name ILIKE 'M%'

inv_no	inv_name	inv_date	inv_amt
101	Mahesh	2024-01-15	20000
102	Mira	2024-02-01	18000
104	Mohan	2024-03-20	15000

Query 3: SELECT fn_investor_count('Mohan')

fn_investor_count('Mohan')
1

Query 4: SELECT fn_investor_count('Unknown')

fn_investor_count('Unknown')
0

Assignment 19- Employee Department

Employee belongs to exactly one department.

Commands :

```
DROP TABLE IF EXISTS employee,
department CASCADE; CREATE TABLE
department (
    deptid INT PRIMARY
    KEY, dname
    VARCHAR(30) NOT
    NULL,
    location VARCHAR(30)
);
CREATE TABLE
employee ( empid
    INT PRIMARY KEY,
    ename VARCHAR(30) NOT NULL,
    phone VARCHAR(15),
    bdate DATE,
    deptid INT REFERENCES department(deptid)
);
INSERT INTO department
VALUES (1,'Computer
science','Pune'),
(2,'Accounts','Mumbai'),
(3,'Library','Pune');
INSERT INTO employee
VALUES
(101,'Sagar','987650101','1998-01-
01',1),
(102,'Riya','987650102','1899-05-10',1),
(103,'Amit','987650103','1997-03-15',2),
(104,'Seema','987650104','2000-11-21',3),
(105,'Pooja','987650105','1996-12-30',1);
SELECT e.*
FROM employee e
JOIN department d ON e.deptid
= d.deptid WHERE d.dname =
'Computer science';
DELETE FROM
employee WHERE
ename ILIKE 'S%';
INSERT INTO department
VALUES (4,'HR','Nashik'),
(5,'Stores','Pimpri');
SELECT e.ename,
d.dname FROM
employee e
JOIN department d ON e.deptid = d.deptid;
CREATE OR REPLACE VIEW
```

```

vw_employee_bdate_before_1900 AS SELECT *
FROM employee
WHERE bdate < '1900-01-01';
CREATE OR REPLACE VIEW
vw_employee_phone AS SELECT
ename, phone
FROM employee;
SELECT * FROM
vw_employee_bdate_before_1900;
SELECT * FROM vw_employee_phone;
CREATE OR REPLACE FUNCTION
fn_employee_department_cursor() RETURNS
TABLE (
    employee_name
    VARCHAR(30), phone
    VARCHAR(15),
    department_name VARCHAR(30)
) AS
$$
DECL
ARE
    rec
    RECORD;
    cur
    CURSOR
    FOR
        SELECT e.ename, e.phone,
            d.dname FROM employee e
            JOIN department d ON e.deptid = d.deptid;
BEGIN
    OPEN
    cur;
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;
        employee_name :
            rec.ename;
        phone :
            rec.phone;
        department_name :
            rec.dname;
        RETURN NEXT;
    END LOOP;
    CLOSE cur;
END;
$$ LANGUAGE plpgsql;
SELECT * FROM fn_employee_department_cursor();

```

Output:

Query 1: SELECT e.* FROM employee e JOIN department d ON e.deptid = d.deptid WHERE d.dname = 'Computer science'

empid	ename	phone	bdate	deptid
101	Sagar	987650101	1998-01-01	1
102	Riya	987650102	1899-05-10	1
105	Pooja	987650105	1996-12-30	1

Query 2: SELECT e.ename, d.dname FROM employee e JOIN department d ON e.deptid = d.deptid

ename	dname
Riya	Computer science
Amit	Accounts
Pooja	Computer science

Query 3: SELECT * FROM vw_employee_bdate_before_1900

empid	ename	phone	bdate	deptid
102	Riya	987650102	1899-05-10	1

Query 4: SELECT * FROM vw_employee_phone

ename	phone
Riya	987650102
Amit	987650103
Pooja	987650105

Query 5: SELECT * FROM fn_employee_department_cursor()

employee_name	phone	department_name
Riya	987650102	Computer science
Amit	987650103	Accounts
Pooja	987650105	Computer science

Assignment 20- Movie Actor Producer

Movie with actor and producer relationships.

Commands :

```
DROP TABLE IF EXISTS movie_actor, movie_producer, producer,  
actor, movie CASCADE; CREATE TABLE movie (  
    movie_id INT  
    PRIMARY KEY,  
    m_name VARCHAR(40)  
    NOT NULL,  
    release_year INT,  
    budget NUMERIC(12,2)  
);  
CREATE TABLE actor (  
    actor_id INT PRIMARY  
    KEY, a_name  
    VARCHAR(30) NOT NULL,  
    age INT,  
    a_address VARCHAR(30)  
);  
CREATE TABLE producer  
( producer_id INT  
    PRIMARY KEY, pname  
    VARCHAR(30) NOT  
    NULL,  
    p_address VARCHAR(30)  
);  
CREATE TABLE movie_actor (  
    movie_id INT REFERENCES  
    movie(movie_id), actor_id INT  
    REFERENCES actor(actor_id),  
    role_name VARCHAR(30),  
    charges NUMERIC(12,2),  
    PRIMARY KEY (movie_id, actor_id)  
);  
CREATE TABLE movie_producer (  
    movie_id INT REFERENCES  
    movie(movie_id), producer_id INT  
    REFERENCES producer(producer_id),  
    PRIMARY KEY (movie_id,  
    producer_id)  
);  
INSERT INTO movie VALUES  
(1,'Chhava',2025,180.00),  
(2,'Lagaan',2001,25.00),  
(3,'Hum Saath Saath Hain',1999,18.00);  
INSERT INTO actor VALUES  
(101,'Vicky Kaushal',36,'Pune'),  
(102,'Madhuri Dixit',57,'Mumbai'),  
(103,'Aamir Khan',59,'Mumbai'),  
(104,'Smita Patil',31,'Pune');
```

```

INSERT INTO producer
VALUES (201,'Maddock
Films','Mumbai'), (202,'Aamir
Productions','Mumbai'),
(203,'Rajshri
Productions','Mumbai');
INSERT INTO movie_actor
VALUES (1,101,'Lead',25.00),
(2,103,'Lead',18.00),
(3,102,'Supporting',12.00),
(3,104,'Lead',10.00);
INSERT INTO
movie_producer VALUES
(1,201),
(2,202),
(3,203);
SELECT MAX(budget) AS
maximum_budget FROM movie;
SELECT
p.pname
FROM
producer p
JOIN movie_producer mp ON p.producer_id =
mp.producer_id JOIN movie m ON m.movie_id =
mp.movie_id
WHERE m.m_name = 'Chhava';
SELECT
a_name
FROM
actor
WHERE a_address = 'Pune';
INSERT INTO producer
VALUES (204,'Blue
Lotus','Pune'),
(205,'Sunrise Media','Delhi');
CREATE OR REPLACE FUNCTION
trg_actor_age_check() RETURNS TRIGGER AS
$$
BEGIN
    IF NEW.age BETWEEN 0 AND 3 THEN
        RAISE EXCEPTION
        'Invalid input'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
before_insert_actor BEFORE
INSERT ON actor
FOR EACH ROW
EXECUTE FUNCTION trg_actor_age_check();

```

```
CREATE OR REPLACE FUNCTION
fn_movies_before_2000() RETURNS TABLE (
    movie_name VARCHAR(40),
    release_year INT,
    budget NUMERIC(12,2)
) AS
$$
BEGIN
    RETURN QUERY
    SELECT m_name, movie.release_year,
    movie.budget FROM movie
    WHERE release_year < 2000
    ORDER BY release_year;
END;
$$ LANGUAGE plpgsql;
```


Output:

Query 1: SELECT MAX(budget) AS maximum_budget FROM movie

maximum_budget
180

Query 2: SELECT p.pname FROM producer p JOIN movie_producer mp ON p.producer_id = mp.producer_id JOIN movie m ON m.movie_id = mp.movie_id WHERE m.m_name = 'Chhava'

pname
Maddock Films

Query 3: SELECT a_name FROM actor WHERE a_address = 'Pune'

a_name
Vicky Kaushal
Smita Patil

Query 4: SELECT * FROM fn_movies_before_2000()

movie_name	release_year	budget
Hum Saath Saath Hain	1999	18

Assignment 21- Book Author

One author writes many books; each book has one author.

Commands :

```
DROP TABLE IF EXISTS book, author CASCADE;
CREATE TABLE author (
    author_id INT
    PRIMARY KEY,
    aname VARCHAR(40) NOT NULL,
    address
    VARCHAR(40),
    phone
    VARCHAR(15)
);
CREATE TABLE book (
    book_id INT PRIMARY
    KEY, btitle
    VARCHAR(60) NOT
    NULL, isbn
    VARCHAR(20)
    UNIQUE,
    category
    VARCHAR(20),
    author_id INT
);
INSERT INTO author VALUES
(1,'J.K. Rowling','London','987650201'),
(2,'George Martin','USA','987650202'),
(3,'Chetan Bhagat','Mumbai','987650203');
INSERT INTO book VALUES
(101,'Harry Potter and the Philosopher Stone','ISBN-
101','Fantasy',1), (102,'A Game of Thrones','ISBN-
102','Fantasy',2),
(103,'Half Girlfriend','ISBN-103','Fiction',3);
INSERT INTO book VALUES
(104,'Harry Potter and the Chamber of Secrets','ISBN-
104','Fantasy',1), (105,'Clash of Kings','ISBN-
105','Fantasy',2);
SELECT a.aname, b.btitle,
b.category FROM author a
JOIN book b ON a.author_id =
b.author_id ORDER BY a.aname;
SELECT b.btitle,
b.category FROM book
b
JOIN author a ON a.author_id = b.author_id
WHERE a.aname = 'J.K. Rowling';
UPDATE author
SET phone = '987654321'
WHERE aname = 'George Martin';
CREATE OR REPLACE FUNCTION
trg_check_book_author() RETURNS TRIGGER
```

```

AS $$
DECLARE
    v_exists
INT; BEGIN
    SELECT COUNT(*)
    INTO
    v_exists
    FROM
    author
    WHERE author_id = NEW.author_id;

END;

IF v_exists = 0 THEN
    RAISE EXCEPTION 'Invalid
Name';
END IF;

RETURN NEW;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
before_insert_book BEFORE
INSERT ON book
FOR EACH ROW
EXECUTE FUNCTION trg_check_book_author();
CREATE OR REPLACE FUNCTION
fn_books_of_author(p_author_name VARCHAR) RETURNS
TABLE (
    book_title
    VARCHAR(60),
    category
    VARCHAR(20)
) AS
DECL
ARE
    rec
    RECORD;
    cur
    CURSOR
    FOR
        SELECT b.btitle, b.category
        FROM book b
        JOIN author a ON a.author_id =
        b.author_id WHERE a.aname ILIKE
        p_author_name;
BEGIN
    OPEN
    cur;
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;

```

```
        book_title      :  
        rec.btitle; category  
        :      rec.category;  
        RETURN NEXT;  
    END LOOP;  
    CLOSE cur;  
END;  
$$ LANGUAGE plpgsql;  
SELECT * FROM fn_books_of_author('J.K. Rowling');
```

Output:

Query 1: SELECT a.aname, b.btitle, b.category FROM author a JOIN book b ON a.author_id = b.author_id ORDER BY a.aname

aname	btitle	category
Chetan Bhagat	Half Girlfriend	Fiction
George Martin	A Game of Thrones	Fantasy
George Martin	Clash of Kings	Fantasy
J.K. Rowling	Harry Potter and the Philosopher Stone	Fantasy
J.K. Rowling	Harry Potter and the Chamber of Secrets	Fantasy

Query 2: SELECT b.btitle, b.category FROM book b JOIN author a ON a.author_id = b.author_id WHERE a.aname = 'J.K. Rowling'

btitle	category
Harry Potter and the Philosopher Stone	Fantasy
Harry Potter and the Chamber of Secrets	Fantasy

Query 3: SELECT * FROM fn_books_of_author('J.K. Rowling')

book_title	category
Harry Potter and the Philosopher Stone	Fantasy
Harry Potter and the Chamber of Secrets	Fantasy

Assignment 22- Bank Database

Bank slip with customer view, average loan and cursor function.

Commands :

```
DROP TABLE IF EXISTS ternary, loan_application,
customer, branch CASCADE; CREATE TABLE branch (
    bid INT PRIMARY KEY,
    brname VARCHAR(30) NOT
    NULL UNIQUE, brcity
    VARCHAR(20) NOT NULL
);
CREATE TABLE
    customer ( cno
    INT PRIMARY
    KEY,
    cname VARCHAR(20)
    NOT NULL, caddr
    VARCHAR(35),
    city VARCHAR(20) NOT NULL
);
CREATE TABLE
    loan_application (
    lno INT PRIMARY
    KEY,
    lamtrequired NUMERIC(12,2) NOT NULL,
    lamtapproved NUMERIC(12,2)
    NOT NULL, ldate DATE NOT
    NULL
);
CREATE TABLE ternary (
    bid INT REFERENCES
    branch(bid), cno INT
    REFERENCES
    customer(cno),
    lno INT REFERENCES
    loan_application(lno), PRIMARY
    KEY (bid, cno, lno)
);
INSERT INTO branch VALUES
(1,'Aundh','Pune'),
(2,'Pimpri','Pune'),
(3,'Deccan','Pune'),
(4,'M.G. ROAD','Pune'),
(5,'J.M.Road','Pune');
INSERT INTO customer
VALUES (101,'Asha','Karve
Nagar','Pune'),
(102,'Amit','Baner','Pune'),
(103,'Pooja','Nigdi','Pune'),
(104,'Rohit','Camp','Pune'),
(105,'Anita','Aundh','Pune'),
(106,'Mohan','Hadapsar','Pune'),
```

```

(107,'Mira','Wakad','Pune');
INSERT INTO
loan_application VALUES
(1001,500000,450000,'2019-
06-10'),
(1002,800000,800000,'2020-01-15'),
(1003,300000,250000,'2020-05-14'),
(1004,1200000,1000000,'2021-04-12'),
(1005,150000,100000,'2019-12-12'),
(1006,250000,250000,'2021-04-12'),
(1007,900000,850000,'2020-07-10'),
(1008,400000,350000,'2021-01-05'),
(1009,220000,200000,'2020-09-25');
INSERT INTO ternary VALUES
(1,101,1001),
(2,102,1002),
(4,103,1003),
(3,104,1004),
(1,105,1005),
(4,106,1006),
(3,102,1007),
(4,107,1008),
(4,103,1009);
SELECT DISTINCT c.cname
FROM customer c
JOIN ternary t ON
c.cno = t.cno JOIN
branch b ON b.bid =
t.bid WHERE b.brname
= 'Aundh';
SELECT MAX(lamtapproved) AS
maximum_approved FROM
loan_application;
SELECT COUNT(*) AS
mg_road_applications FROM
ternary t
JOIN branch b ON b.bid = t.bid
WHERE b.brname = 'M.G. ROAD';
SELECT * FROM branch;
CREATE OR REPLACE VIEW
vw_customer_names AS SELECT
DISTINCT cname
FROM customer;
CREATE OR REPLACE VIEW
vw_average_loan_amount AS SELECT
AVG(lamtapproved) AS
average_approved_amount FROM
loan_application;
SELECT * FROM
vw_customer_names; SELECT *
FROM
vw_average_loan_amount;
CREATE OR REPLACE FUNCTION

```

```

fn_customer_branch_by_loan_date(p_loan_date DATE) RETURNS
TABLE (
    customer_name VARCHAR(20),
    branch_name VARCHAR(30)
) AS
$$
DECLAR
E
    rec RECORD;
    cur CURSOR FOR
        SELECT c.cname,
            b.brname FROM customer
            c
            JOIN ternary t ON c.cno =
            t.cno JOIN branch b ON b.bid
            = t.bid
            JOIN loan_application l ON l.lno =
            t.lno WHERE l.ldate =
            p_loan_date;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;
        customer_name :
            rec.cname;
        branch name :

```


Output:

Query 1: SELECT DISTINCT c.cname FROM customer c JOIN ternary t ON c.cno = t.cno JOIN branch b ON b.bid = t.bid WHERE b.brname = 'Aundh'

cname
Asha
Anita

Query 2: SELECT MAX(lamtapproved) AS maximum_approved FROM loan_application

maximum_approved
1000000

Query 3: SELECT COUNT(*) AS mg_road_applications FROM ternary t JOIN branch b ON b.bid = t.bid WHERE b.brname = 'M.G. ROAD'

mg_road_applications
4

Query 4: SELECT * FROM branch

bid	brname	brcity
1	Aundh	Pune
2	Pimpri	Pune
3	Deccan	Pune
4	M.G. ROAD	Pune
5	J.M.Road	Pune

Query 5: SELECT * FROM vw_customer_names

cname
Asha
Amit
Pooja
Rohit
Anita
Mohan
Mira

Query 6: SELECT * FROM vw_average_loan_amount

average_approved_amount
472222.2222222225

Query 7: SELECT * FROM fn_customer_branch_by_loan_date('2021-04-12')

customer_name	branch_name
Rohit	Deccan
Mohan	M.G. ROAD

Assignment 23- Patient Doctor

A doctor can have many patients; each patient belongs to one doctor.

Commands :

```
DROP TABLE IF EXISTS patient,
doctor CASCADE; CREATE TABLE
doctor (
    did INT PRIMARY KEY,
    dname VARCHAR(30) NOT NULL,
    specialization VARCHAR(30)
);
CREATE TABLE
patient ( pid
INT PRIMARY
KEY,
pname VARCHAR(40) NOT NULL,
age INT,
did INT REFERENCES doctor(did)
);
INSERT INTO doctor VALUES
(1,'Dr. Sharma','General
Medicine'), (2,'Dr.
Gupta','Neurology'),
(3,'Dr. Agarwal','Orthopedic'),
(4,'Dr. Joshi','Cardiology');
INSERT INTO patient VALUES
(101,'Mr.P.V.Joshi',62,4),
(102,'Mrs. Rekha',45,1),
(103,'Master Rohan',11,1),
(104,'Mr. Patil',50,2),
(105,'Ms. Neha',29,3);
SELECT
p.pname
FROM
patient p
JOIN doctor d ON p.did = d.did
WHERE d.dname = 'Dr.
Sharma'; SELECT *
FROM doctor;
UPDATE doctor
SET specialization =
'Cardiologist' WHERE dname
= 'Dr. Gupta';
DELETE FROM patient
WHERE did = (SELECT did FROM doctor WHERE dname = 'Dr. Agarwal');
CREATE OR REPLACE VIEW
vw_doctor_of_pv_joshi AS SELECT d.*
```

```

FROM doctor d
JOIN patient p ON d.did
= p.did WHERE p.pname
= 'Mr.P.V.Joshi';
CREATE OR REPLACE VIEW
vw_patient_name_age AS SELECT
pname, age
FROM patient;
SELECT * FROM
vw_doctor_of_pv_joshi; SELECT
* FROM vw_patient_name_age;
CREATE OR REPLACE FUNCTION
fn_doctor_patient_cursor() RETURNS TABLE
(
    doctor_name VARCHAR(30),
    specialization
    VARCHAR(30),
    patient_name
    VARCHAR(40)
) AS
$$
DECL
ARE
    rec RECORD;
    cur CURSOR FOR
        SELECT d.dname, d.specialization,
        p.pname FROM doctor d
        JOIN patient p ON d.did = p.did;
BEGIN
    OPEN
    cur;
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;
        doctor_name :
            rec.dname;
        specialization :
            rec.specializati
on; patient_name :
            rec.pname;

        RETURN
    NEXT; END
    LOOP;
    CLOSE cur;
END;
$$ LANGUAGE plpgsql;
SELECT * FROM fn_doctor_patient_cursor();

```

Output:

Query 1: SELECT p.pname FROM patient p JOIN doctor d ON p.did = d.did WHERE d.dname = 'Dr. Sharma'

pname
Mrs. Rekha
Master Rohan

Query 2: SELECT * FROM doctor

did	dname	specialization
1	Dr. Sharma	General Medicine
2	Dr. Gupta	Neurology
3	Dr. Agarwal	Orthopedic
4	Dr. Joshi	Cardiology

Query 3: SELECT * FROM vw_doctor_of_pv_joshi

did	dname	specialization
4	Dr. Joshi	Cardiology

Query 4: SELECT * FROM vw_patient_name_age

pname	age
Mr.P.V.Joshi	62
Mrs. Rekha	45
Master Rohan	11
Mr. Patil	50

Query 5: SELECT * FROM fn_doctor_patient_cursor()

doctor_name	specialization	patient_name
Dr. Joshi	Cardiology	Mr.P.V.Joshi
Dr. Sharma	General Medicine	Mrs. Rekha
Dr. Sharma	General Medicine	Master Rohan
Dr. Gupta	Cardiologist	Mr. Patil

Assignment 24- Product Order

Products and Orders with many-to-many relationship through order items.

Commands :

```
DROP TABLE IF EXISTS order_item, orders,
```

```
product CASCADE; CREATE TABLE product (  
    pid INT PRIMARY KEY,  
    pname VARCHAR(30)  
    NOT NULL, price  
    NUMERIC(10,2)  
);
```

```
CREATE TABLE  
    orders ( oid  
    INT PRIMARY  
    KEY,  
    orderdate  
    DATE, customer  
    VARCHAR(30)  
);
```

```
CREATE TABLE order_item (  
    oid INT REFERENCES  
    orders(oid), pid INT  
    REFERENCES  
    product(pid), qty INT,  
    PRIMARY KEY (oid, pid)  
);
```

```
INSERT INTO product VALUES  
(1,'Keyboard',900.00),  
(2,'Mouse',450.00),  
(3,'Monitor',6500.00),  
(4,'Printer',7200.00);
```

```
INSERT INTO  
orders VALUES  
(101,'2024-01-  
10','Asha'),  
(102,'2024-01-11','Rohit'),  
(103,'2024-01-12','Pooja');
```

```
INSERT INTO order_item  
VALUES (101,1,3),  
(101,2,2),  
(102,3,1),  
(103,4,2),  
(103,1,1);
```

```
SELECT p.*  
FROM product p  
JOIN order_item oi ON p.pid  
= oi.pid WHERE oi.oid =  
101;  
SELECT * FROM product;  
SELECT pname,  
price FROM
```

```

product;
INSERT INTO product VALUES
(5,'Scanner',4800.00),
(6,'Speaker',2200.00);
CREATE OR REPLACE VIEW
vw_product_details AS SELECT * FROM
product;
CREATE OR REPLACE VIEW
vw_highest_demand_order AS SELECT o.oid,
o.customer, SUM(oi.qty) AS total_qty
FROM orders o
JOIN order_item oi ON o.oid
= oi.oid GROUP BY o.oid,
o.customer
ORDER BY total_qty
DESC LIMIT 1;
SELECT * FROM
vw_product_details; SELECT *
FROM
vw_highest_demand_order;
CREATE OR REPLACE FUNCTION
fn_product_with_customer() RETURNS TABLE
(
    product_name
    VARCHAR(30), price
    NUMERIC(10,2),
    customer VARCHAR(30)
) AS
$$
BEGIN
N
    RETURN QUERY
    SELECT p.pname, p.price,
    o.customer FROM product p
    JOIN order_item oi ON p.pid = oi.pid
    JOIN orders o ON o.oid =
    oi.oid ORDER BY
    o.customer, p.pname;
END;
$$ LANGUAGE plpgsql;
SELECT * FROM fn_product_with_customer();

```

Output:

Query 1: SELECT p.* FROM product p JOIN order_item oi ON p.pid = oi.pid WHERE oi.oid = 101

pid	pname	price
1	Keyboard	900
2	Mouse	450

Query 2: SELECT * FROM product

pid	pname	price
1	Keyboard	900
2	Mouse	450
3	Monitor	6500
4	Printer	7200

Query 3: SELECT pname, price FROM product

pname	price
Keyboard	900
Mouse	450
Monitor	6500
Printer	7200

Query 4: SELECT * FROM vw_product_details

pid	pname	price
1	Keyboard	900
2	Mouse	450
3	Monitor	6500
4	Printer	7200
5	Scanner	4800
6	Speaker	2200

Query 5: SELECT * FROM vw_highest_demand_order

oid	customer	total_qty
101	Asha	5

Query 6: SELECT * FROM fn_product_with_customer()

product_name	price	customer
Keyboard	900	Asha
Mouse	450	Asha
Keyboard	900	Pooja
Printer	7200	Pooja
Monitor	6500	Rohit

Assignment 25- Student Competition

Student and competition with registration-number function.

Commands :

```
DROP TABLE IF EXISTS student_competition, competition,
student CASCADE; CREATE TABLE student (
    sreg_no INT
    PRIMARY KEY, sname
    VARCHAR(30) NOT
    NULL,
    class VARCHAR(10) NOT NULL CHECK (class IN ('FYBCA','SYBCA','TYBCA'))
);
CREATE TABLE
    competition ( cno
    INT PRIMARY
    KEY,
    cname VARCHAR(30)
    NOT NULL, c_type
    VARCHAR(20) NOT
    NULL
);
CREATE TABLE student_competition (
    sreg_no INT REFERENCES
    student(sreg_no), cno INT
    REFERENCES competition(cno),
    rank INT,
    year INT,
    PRIMARY KEY (sreg_no, cno)
);
INSERT INTO student VALUES
(101,'Aarav','SYBCA'),
(102,'Pooja','SYBCA'),
(103,'Rohan','FYBCA'),
(104,'Aditi','TYBCA'),
(105,'Ankita','SYBCA');
INSERT INTO competition
VALUES (11,'Quiz','Academic'),
(12,'Football','Sports'),
(13,'Coding','Technical'),
(14,'Debate','Cultural');
INSERT INTO
student_competition VALUES
(101,11,2,2024),
(102,11,1,2024),
(102,12,2,2024),
(102,13,3,2024),
(103,12,4,2024),
(104,13,1,2024),
(105,11,3,2024);
SELECT DISTINCT s.sname
FROM student s
```



```
JOIN student_competition sc ON
s.sreg_no = sc.sreg_no JOIN competition
c ON c.cno = sc.cno
WHERE c.cname = 'Quiz';
SELECT cname
FROM competition;
SELECT *
FROM student
WHERE sname ILIKE 'A%';
INSERT INTO student VALUES
(106,'Nikita','FYBCA'),
(107,'Pratik','TYBCA');
CREATE OR REPLACE FUNCTION
trg_competition_welcome() RETURNS
TRIGGER AS $$
DECLARE
    v_count INT;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM
        student_competition
    WHERE sreg_no =
        NEW.sreg_no;
END;
```

```

IF v_count > 2 THEN
    RAISE NOTICE 'All students are welcome for
competition'; END IF;
RETURN NEW;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
after_insert_student_competition
AFTER INSERT ON
student_competition
FOR EACH ROW
EXECUTE FUNCTION trg_competition_welcome();
CREATE OR REPLACE FUNCTION
fn_competitions_of_student_reg(p_reg_no INT) RETURNS
TABLE (
    student_name VARCHAR(30),
    competition_name
    VARCHAR(30)
) AS
$$
DECL
ARE
    rec RECORD;
    cur CURSOR FOR
        SELECT s.sname,
               c.cname

               FROM Student S
               JOIN student_competition sc ON s.sreg_no = sc.sreg_no
               JOIN competition c ON c.cno = sc.cno
               WHERE s.sreg_no = p_reg_no;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;
        student_name :
            rec.sname;
        competition_name :
            rec.cname;

        RETURN NEXT;
    END LOOP;

```

Output:

Query 1: SELECT DISTINCT s.sname FROM student s JOIN student_competition sc ON s.sreg_no = sc.sreg_no JOIN competition c ON c.cno = sc.cno WHERE c.cname = 'Quiz'

sname
Aarav
Pooja
Ankita

Query 2: SELECT cname FROM competition

cname
Quiz
Football
Coding
Debate

Query 3: SELECT * FROM student WHERE sname ILIKE 'A%'

sreg_no	sname	class
101	Aarav	SYBCA
104	Aditi	TYBCA
105	Ankita	SYBCA

Query 4: SELECT * FROM fn_competitions_of_student_reg(102)

student_name	competition_name
Pooja	Quiz
Pooja	Football
Pooja	Coding

Assignment 26- Property Owner

One owner can own many properties; each property has one owner.

Commands :

```
DROP TABLE IF EXISTS property,
owner CASCADE; CREATE TABLE
owner (
    owner_id INT
    PRIMARY KEY, oname
    VARCHAR(30) NOT
    NULL,
    address
    VARCHAR(30),
    phone
    VARCHAR(15)
);
CREATE TABLE
property ( pno
INT PRIMARY
KEY,
description
VARCHAR(50), area
VARCHAR(20),
owner_id INT REFERENCES owner(owner_id)
);
INSERT INTO owner VALUES
(1,'Mr. Kadam','Pune','987650301'),
(2,'Mr. Patil','Pimpri','987650302'),
(3,'Mrs. Joshi','Kothrud','987650303');
INSERT INTO property
VALUES (101,'2BHK
Flat','Moshi',1),
(102,'Shop','Kothrud',2),
(103,'Plot','Hadapsar',3),
(104,'Bungalow','Moshi',1);
SELECT *
FROM property
WHERE area =

'Moshi'; SELECT *

FROM owner;

SELECT p.*
FROM property p
JOIN owner o ON p.owner_id =
o.owner_id WHERE o.oname =
'Mr. Kadam';
UPDATE owner
SET phone = '987842621'
WHERE oname = 'Mr. Patil';
```

```

CREATE OR REPLACE FUNCTION
trg_property_area_check() RETURNS
TRIGGER AS $$
BEGIN
    IF NEW.area NOT IN
        ('Moshi','Kothrud','Hadapsar','Baner','Aundh','Pimpri') THEN
        RAISE EXCEPTION 'Area should be in Pune';
    END IF;
    RETURN
    NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
before_insert_property BEFORE
INSERT ON property
FOR EACH ROW
EXECUTE FUNCTION trg_property_area_check();
CREATE OR REPLACE FUNCTION
fn_owner_cursor() RETURNS TABLE (
    owner_name
    VARCHAR(30),
    address
    VARCHAR(30),
    phone
    VARCHAR(15)
) AS
$$
DECL
ARE
    rec RECORD;
    cur CURSOR FOR
        SELECT oname, owner.address,
            owner.phone FROM owner;
BEGIN
    OPEN
    cur;
    LOOP
        FETCH cur INTO
        rec; EXIT WHEN
        NOT FOUND;
        owner_name :
            rec.oname;
        address :
            rec.address;
        phone : rec.phone;
        RETURN NEXT;
    END LOOP;
    CLOSE cur;
END;
$$ LANGUAGE plpgsql;
SELECT * FROM fn_owner_cursor();

```

Output:

Query 1: SELECT * FROM property WHERE area = 'Moshi'

pno	description	area	owner_id
101	2BHK Flat	Moshi	1
104	Bungalow	Moshi	1

Query 2: SELECT * FROM owner

owner_id	oname	address	phone
1	Mr. Kadam	Pune	987650301
2	Mr. Patil	Pimpri	987650302
3	Mrs. Joshi	Kothrud	987650303

Query 3: SELECT p.* FROM property p JOIN owner o ON p.owner_id = o.owner_id WHERE o.oname = 'Mr. Kadam'

pno	description	area	owner_id
101	2BHK Flat	Moshi	1
104	Bungalow	Moshi	1

Query 4: SELECT * FROM fn_owner_cursor()

owner_name	address	phone
Mr. Kadam	Pune	987650301
Mr. Patil	Pimpri	987842621
Mrs. Joshi	Kothrud	987650303

Assignment 27- Movie Actor Producer

Movie-Actor many-to-many and Movie-Producer many-to-one version.

Commands :

```
DROP TABLE IF EXISTS movie_actor, movie, actor,
producer CASCADE; CREATE TABLE producer (
    producer_id INT PRIMARY
    KEY, pname VARCHAR(30)
    NOT NULL,
    p_address VARCHAR(30)
);
CREATE TABLE movie (
    movie_id INT
    PRIMARY KEY,
    m_name VARCHAR(30)
    NOT NULL,
    release_year
    INT, budget
    NUMERIC(12,2),
    producer_id INT REFERENCES producer(producer_id)
);
CREATE TABLE actor (
    actor_id INT PRIMARY
    KEY, a_name
    VARCHAR(30) NOT
    NULL, city
    VARCHAR(30)
);
CREATE TABLE movie_actor (
    movie_id INT REFERENCES
    movie(movie_id), actor_id INT
    REFERENCES actor(actor_id),
    charges NUMERIC(12,2),
    PRIMARY KEY (movie_id, actor_id)
);
INSERT INTO producer
VALUES (1,'Raj
Films','Mumbai'),
(2,'Dream Studio','Delhi');
INSERT INTO movie
VALUES (101,'Mission
2015',2015,120.00,1),
(102,'Family
Drama',2014,45.00,1),
(103,'Action
Storm',2016,150.00,2);
INSERT INTO actor VALUES
(11,'Amitabh
Bachchan','Mumbai'),
(12,'Madhuri Dixit','Delhi'),
(13,'Rajkummar Rao','Pune'),
```

```

(14,'Vidya Balan','Mumbai');
INSERT INTO movie_actor
VALUES (101,11,12.00),
(102,11,10.00),
(103,12,14.00),
(101,14,8.00);
SELECT *
FROM movie
WHERE release_year = 2015;
SELECT
a_name
FROM
actor
WHERE city = 'Mumbai';
SELECT m_name,
budget FROM
movie;
INSERT INTO actor VALUES
(15,'Ayushmann
Khurrana','Chandigarh'),
(16,'Kangana
Ranaut','Manali');
CREATE OR REPLACE FUNCTION
trg_movie_year_check() RETURNS TRIGGER
AS $$
BEGIN
    IF NEW.release_year > EXTRACT(YEAR FROM CURRENT_DATE) THEN
        RAISE EXCEPTION 'Release year should not be greater
        than current year'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
before_insert_movie BEFORE
INSERT ON movie
FOR EACH ROW
EXECUTE FUNCTION trg_movie_year_check();
CREATE OR REPLACE FUNCTION
fn_amitabh_movie_charges() RETURNS
TABLE (
    movie_name
    VARCHAR(30), charges
    NUMERIC(12,2)
) AS
$$
DECL
ARE
    rec
    RECORD;
    cur
    CURSOR
    FOR

```



```
SELECT m.m_name, ma.charges
FROM movie m
JOIN movie_actor ma ON m.movie_id =
ma.movie_id JOIN actor a ON
a.actor_id = ma.actor_id
WHERE a.a_name = 'Amitabh Bachchan';

BEGIN
  OPEN
  cur;
  LOOP
    FETCH cur INTO rec;
    EXIT WHEN NOT FOUND;
    movie_name :
      rec.m_name;
    charges : rec.charges;
    RETURN NEXT;
  END LOOP;
  CLOSE cur;
END;
$$ LANGUAGE plpgsql;
```

Output:

Query 1: SELECT * FROM movie WHERE release_year = 2015

movie_id	m_name	release_year	budget	producer_id
101	Mission 2015	2015	120	1

Query 2: SELECT a_name FROM actor WHERE city = 'Mumbai'

a_name
Amitabh Bachchan
Vidya Balan

Query 3: SELECT m_name, budget FROM movie

m_name	budget
Mission 2015	120
Family Drama	45
Action Storm	150

Query 4: SELECT * FROM fn_amitabh_movie_charges()

movie_name	charges
Mission 2015	12
Family Drama	10

Assignment 28- Student Course Enrollment

Student, Course and Enrollment with count function and view.

Commands :

```
DROP TABLE IF EXISTS enrollment, course,
student CASCADE; CREATE TABLE student (
    student_id INT
    PRIMARY KEY, name
    VARCHAR(30) NOT
    NULL,
    age INT
);
CREATE TABLE course (
    course_id INT
    PRIMARY KEY,
    course_name VARCHAR(40)
    NOT NULL, credits INT
);
CREATE TABLE enrollment (
    student_id INT REFERENCES
    student(student_id), course_id INT
    REFERENCES course(course_id),
    enrollment_date DATE,
    PRIMARY KEY (student_id, course_id)
);
INSERT INTO student
VALUES (1,'Ram',20),
(2,'Pooja',21),
(3,'Sneha',19),
(4,'Rohit',22);
INSERT INTO course VALUES
(101,'B.Sc(Comp.Sci)',4),
(102,'B.C.A',5),
(103,'Statistics',3),
(104,'AI Basics',4);
INSERT INTO
enrollment VALUES
(1,101,'2024-06-01'),
(1,102,'2024-06-15'),
(2,101,'2024-06-03'),
(3,103,'2024-06-07'),
(4,104,'2024-06-09');
SELECT s.name
FROM student s
JOIN enrollment e ON s.student_id =
e.student_id JOIN course c ON
c.course_id = e.course_id
WHERE c.course_name = 'B.Sc(Comp.Sci)';
SELECT *
FROM course
WHERE credits > 3;
```

```
INSERT INTO course VALUES
(105,'Cyber
Security',4),
(106,'Data
Mining',5);
SELECT
e.enrollment_date
FROM enrollment e
JOIN student s ON s.student_id =
e.student_id WHERE s.name =
'Ram';
CREATE OR REPLACE FUNCTION
fn_student_count() RETURNS INT AS
$$
DECLARE
    v_count INT;
BEGIN
    SELECT COUNT(*) INTO v_count FROM student;
    RETURN v_count;
END;
$$ LANGUAGE

plpgsql; SELECT

fn_student_count();

CREATE OR REPLACE VIEW
vw_student_course_names AS SELECT
s.name, c.course_name
FROM student s
JOIN enrollment e ON s.student_id =
e.student_id JOIN course c ON
c.course_id = e.course_id
ORDER BY s.name, c.course_name;
SELECT * FROM vw_student_course_names;
```

Output:

Query 1: SELECT s.name FROM student s JOIN enrollment e ON s.student_id = e.student_id JOIN course c ON c.course_id = e.course_id WHERE c.course_name = 'B.Sc(Comp.Sci)'

name
Ram
Pooja

Query 2: SELECT * FROM course WHERE credits > 3

course_id	course_name	credits
101	B.Sc(Comp.Sci)	4
102	B.C.A	5
104	AI Basics	4

Query 3: SELECT e.enrollment_date FROM enrollment e JOIN student s ON s.student_id = e.student_id WHERE s.name = 'Ram'

enrollment_date
2024-06-01
2024-06-15

Query 4: SELECT fn_student_count()

fn_student_count()
4

Query 5: SELECT * FROM vw_student_course_names

name	course_name
Pooja	B.Sc(Comp.Sci)
Ram	B.C.A
Ram	B.Sc(Comp.Sci)
Rohit	AI Basics
Sneha	Statistics

Assignment 29- Bus Route Driver

Bus-driver simplified relationship with function and delete trigger.

Commands :

```
DROP TABLE IF EXISTS driver, bus, route

CASCADE; CREATE TABLE route (
    route_no INT
    PRIMARY KEY, source
    VARCHAR(20),
    destination
    VARCHAR(20),
    no_of_stations INT
);
CREATE TABLE bus (
    bus_no INT PRIMARY
    KEY, b_capacity INT
    NOT NULL,
    depot_name
    VARCHAR(20),
    route_no INT REFERENCES route(route_no)
);
CREATE TABLE driver
( driver_no INT
  PRIMARY KEY,
  driver_name VARCHAR(20)
  NOT NULL, license_no INT
  UNIQUE NOT NULL, address
  VARCHAR(20),
  d_age INT,
  salary NUMERIC(10,2),
  bus_no INT REFERENCES bus(bus_no)
);
INSERT INTO route VALUES
(1,'Delhi','Jaipur',12),
(2,'Delhi','Agra',10),
(3,'Pune','Mumbai',18),
(4,'Kothrud','Swargate',9);
INSERT INTO bus VALUES
(31,55,'Kothrud',1),
(32,60,'Pune Station',2),
(33,45,'Kothrud',3),
(34,70,'Kothrud',4);
INSERT INTO driver VALUES
(301,'Ajay',7101,'Pune',54,32000,
31),
(302,'Bharat',7102,'Satara',49,22000,33),
(303,'Chetan',7103,'Pune',61,28000,34),
(304,'Deepak',7104,'Nashik',52,26000,32),
(305,'Farhan',7105,'Pune',42,30000,31);
SELECT *
FROM bus
WHERE b_capacity > 50;
```

```

SELECT *
FROM route
WHERE source = 'Delhi';
SELECT driver_name,
license_no FROM driver;
SELECT *
FROM driver
WHERE d_age > 50;
CREATE OR REPLACE FUNCTION
fn_driver_details_by_depot(p_depot_name VARCHAR) RETURNS
TABLE (
    driver_name VARCHAR(20),
    age INT,
    salary NUMERIC(10,2)
) AS
$$
BEGIN
    RETURN QUERY
    SELECT d.driver_name,
    d.d_age, d.salary FROM driver
    d
    JOIN bus b ON d.bus_no = b.bus_no
    WHERE b.depot_name ILIKE
        p_depot_name AND d.d_age
        > 50;
END;
$$ LANGUAGE plpgsql;
SELECT * FROM fn_driver_details_by_depot('Kothrud');
CREATE OR REPLACE FUNCTION
trg_prevent_delete_pune_driver() RETURNS
TRIGGER AS $$
BEGIN
    IF OLD.address = 'Pune' THEN
        RAISE EXCEPTION 'Deletion not allowed for
        drivers living in Pune'; END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
before_delete_driver BEFORE
DELETE ON driver
FOR EACH ROW
EXECUTE FUNCTION trg_prevent_delete_pune_driver();

```

Output:

Query 1: SELECT * FROM bus WHERE b_capacity > 50

bus_no	b_capacity	depot_name	route_no
31	55	Kothrud	1
32	60	Pune Station	2
34	70	Kothrud	4

Query 2: SELECT * FROM route WHERE source = 'Delhi'

route_no	source	destination	no_of_stations
1	Delhi	Jaipur	12
2	Delhi	Agra	10

Query 3: SELECT driver_name, license_no FROM driver

driver_name	license_no
Ajay	7101
Bharat	7102
Chetan	7103
Deepak	7104
Farhan	7105

Query 4: SELECT * FROM driver WHERE d_age > 50

driver_no	driver_name	license_no	address	d_age	salary	bus_no
301	Ajay	7101	Pune	54	32000	31
303	Chetan	7103	Pune	61	28000	34
304	Deepak	7104	Nashik	52	26000	32

Query 5: SELECT * FROM fn_driver_details_by_depot('Kothrud')

driver_name	age	salary
Ajay	54	32000
Chetan	61	28000

Assignment 30- Branch Customer Loan Application

Bank slip with sanctioned loan totals and customer multi-branch function.

Commands :

```
DROP TABLE IF EXISTS loan_application, customer,
```

```
branch CASCADE; CREATE TABLE branch (  
    bid INT PRIMARY KEY,  
    brname VARCHAR(30) NOT NULL,  
    brcity VARCHAR(20)  
);
```

```
CREATE TABLE  
    customer ( cno  
    INT PRIMARY  
    KEY,  
    cname VARCHAR(20)  
    NOT NULL, caddr  
    VARCHAR(35),  
    city VARCHAR(20)  
);
```

```
CREATE TABLE  
    loan_application (  
    lno INT PRIMARY  
    KEY,  
    lamtrequired NUMERIC(12,2),  
    lamtapproved  
    NUMERIC(12,2), ldate  
    DATE,  
    bid INT REFERENCES  
    branch(bid), cno INT  
    REFERENCES customer(cno)  
);
```

```
INSERT INTO branch VALUES  
(1,'Deccan','Pune'),  
(2,'M.G.ROAD','Pune'),  
(3,'Aundh','Pune');
```

```
INSERT INTO customer  
VALUES  
(101,'Amit','Baner','Pune'),  
(102,'Pooja','Nigdi','Pune'),  
(103,'Rohit','Camp','Pune'),  
(104,'Anita','Aundh','Pune');
```

```
INSERT INTO loan_application  
VALUES  
(1001,500000,450000,'2024-01-  
10',1,101),  
(1002,400000,350000,'2024-01-11',2,102),  
(1003,300000,300000,'2024-01-12',2,103),  
(1004,700000,650000,'2024-01-13',2,104),  
(1005,600000,550000,'2024-01-14',1,102),  
(1006,200000,180000,'2024-01-15',3,101);
```

```
SELECT  
c.cname
```

```

FROM
customer c
JOIN loan_application l ON
c.cno = l.cno WHERE
l.lamtapproved <
l.lamtrequired;
SELECT MAX(lamtapproved) AS
max_approved FROM
loan_application;
SELECT SUM(l.lamtapproved) AS
total_sanctioned FROM
loan_application l
JOIN branch b ON
b.bid = l.bid WHERE
b.brname = 'Deccan';
SELECT COUNT(*) AS
mg_road_count FROM
loan_application l
JOIN branch b ON b.bid
= l.bid WHERE
b.brname = 'M.G.ROAD';
CREATE OR REPLACE VIEW
vw_less_than_required_loan AS SELECT
c.*, l.lno, l.lamtrequired, l.lamtapproved
FROM customer c
JOIN loan_application l ON
c.cno = l.cno WHERE
l.lamtapproved <
l.lamtrequired;
SELECT * FROM vw_less_than_required_loan;
CREATE OR REPLACE FUNCTION
fn_customers_with_multiple_branches() RETURNS
TABLE (
    customer_name
    VARCHAR(20), branch_count
    BIGINT
) AS
$$
BEGIN
    RETURN QUERY
    SELECT c.cname, COUNT(DISTINCT l.bid)
    FROM customer c
    JOIN loan_application l ON
    c.cno = l.cno GROUP BY
    c.cname
    HAVING COUNT(DISTINCT l.bid) > 1;
END;
$$ LANGUAGE plpgsql;
SELECT * FROM fn_customers_with_multiple_branches();

```

Output:

Query 1: SELECT c.cname FROM customer c JOIN loan_application l ON c.cno = l.cno WHERE l.lamtapproved < l.lamtrequired

cname
Amit
Pooja
Anita
Pooja
Amit

Query 2: SELECT MAX(lamtapproved) AS max_approved FROM loan_application

max_approved
650000

Query 3: SELECT SUM(l.lamtapproved) AS total_sanctioned FROM loan_application l JOIN branch b ON b.bid = l.bid WHERE b.brname = 'Deccan'

total_sanctioned
1000000

Query 4: SELECT COUNT(*) AS mg_road_count FROM loan_application l JOIN branch b ON b.bid = l.bid WHERE b.brname = 'M.G.ROAD'

mg_road_count
3

Query 5: SELECT * FROM vw_less_than_required_loan

cno	cname	caddr	city	lno	lamtrequired	lamtapproved
101	Amit	Baner	Pune	1001	500000	450000
102	Pooja	Nigdi	Pune	1002	400000	350000
104	Anita	Aundh	Pune	1004	700000	650000
102	Pooja	Nigdi	Pune	1005	600000	550000
101	Amit	Baner	Pune	1006	200000	180000

Query 6: SELECT * FROM fn_customers_with_multiple_branches()

customer_name	branch_count
Amit	2
Pooja	2

Assignment 31- Salesman Trip Dept Expense

Trips belong to salesmen and expenses belong to trips.

Commands :

```
DROP TABLE IF EXISTS expense, trip,
salesman, dept CASCADE; CREATE TABLE dept
(
    dept_no VARCHAR(10) PRIMARY KEY,
    dept_name VARCHAR(20)
);
CREATE TABLE
    salesman ( sno
    INT PRIMARY
    KEY,
    s_name VARCHAR(30) NOT NULL,
    start_year INT,
    dept_no VARCHAR(10) REFERENCES dept(dept_no)
);
CREATE TABLE trip (
    tno INT PRIMARY KEY,
    from_city
    VARCHAR(20), to_city
    VARCHAR(20),
    departure_date
    DATE, return_date
    DATE,
    sno INT REFERENCES salesman(sno)
);
CREATE TABLE
    expense ( eid
    INT PRIMARY
    KEY, amount
    NUMERIC(10,2),
    tno INT REFERENCES trip(tno)
);
INSERT INTO dept
VALUES
('D1','Sales'),
('D2','Marketing'),
('D3','Support');
INSERT INTO salesman VALUES
(1,'Amit',2018,'D1'),
(2,'Pooja',2019,'D2'),
(3,'Rohit',2020,'D1');
INSERT INTO trip VALUES
(101,'Mumbai','Pune','2024-01-10','2024-01-12',1),
(102,'Pune','Mumbai','2024-01-15','2024-01-16',1),
(103,'Delhi','Mumbai','2024-01-18','2024-01-22',2),
(104,'Pune','Mumbai','2024-02-01','2024-02-03',3);
INSERT INTO expense VALUES
(1001,800.00,101),
```

```
(1002,1600.00,102),  
(1003,2500.00,103),  
(1004,1400.00,104);
```

```
SELECT * FROM
```

```
salesman; SELECT *
```

```
FROM trip
```

```
WHERE from_city = 'Mumbai';
```

```
SELECT
```

```
dept_name
```

```
FROM dept;
```

```
SELECT *
```

```
FROM expense
```

```
WHERE amount > 1000;
```

```
CREATE OR REPLACE FUNCTION
```

```
fn_trip_with_max_expense() RETURNS
```

```
TABLE (
```

```
    trip_no INT,
```

```
    from_city
```

```
    VARCHAR(20),
```

```
    to_city
```

```
    VARCHAR(20),
```

```
    amount
```

```
    NUMERIC(10,2)
```

```
) AS
```

```
$$
```

```
DECL
```

```
ARE
```

```
    rec RECORD;
```

```
    cur CURSOR FOR
```

```
        SELECT t.tno, t.from_city, t.to_city,
```

```
        e.amount FROM trip t
```

```
        JOIN expense e ON t.tno
```

```
        = e.tno ORDER BY
```

```
        e.amount DESC
```

```
        LIMIT 1;
```

```
BEGIN
```

```
    OPEN
```

```
    cur;
```

```
    LOOP
```

```
END;
```

```
FETCH cur INTO rec;
EXIT WHEN NOT FOUND;
trip_no :    rec.tno;
from_city :  rec.from_city;
to_city :   rec.to_city;
amount : rec.amount;
RETURN NEXT;
END LOOP;
CLOSE cur;
$$ LANGUAGE plpgsql;
SELECT * FROM fn_trip_with_max_expense();
CREATE OR REPLACE FUNCTION fn_pune_to_mumbai_trip_count()
RETURNS INT AS $$
DECLARE
    v_count INT;
BEGIN
    SELECT COUNT(*) INTO
    v_count FROM trip
    WHERE from_city = 'Pune'
    AND to_city =
    'Mumbai'; RETURN
    v_count;
END;
$$ LANGUAGE plpgsql;
```

Output:

Query 1: **SELECT * FROM salesman**

sno	s_name	start_year	dept_no
1	Amit	2018	D1
2	Pooja	2019	D2
3	Rohit	2020	D1

Query 2: **SELECT * FROM trip WHERE from_city = 'Mumbai'**

tno	from_city	to_city	departure_date	return_date	sno
101	Mumbai	Pune	2024-01-10	2024-01-12	1

Query 3: **SELECT dept_name FROM dept**

dept_name
Sales
Marketing
Support

Query 4: **SELECT * FROM expense WHERE amount > 1000**

eid	amount	tno
1002	1600	102
1003	2500	103
1004	1400	104

Query 5: **SELECT * FROM fn_trip_with_max_expense()**

trip_no	from_city	to_city	amount
103	Delhi	Mumbai	2500

Query 6: **SELECT fn_pune_to_mumbai_trip_count()**

fn_pune_to_mumbai_trip_count()
2

Assignment 32- Train Passenger Ticket

Train to ticket and passenger to ticket one-to-many interpretation.

Commands :

```
DROP TABLE IF EXISTS ticket,  
passenger, train CASCADE; CREATE
```

```
TABLE train (  
    train_no INT PRIMARY  
    KEY, train_name  
    VARCHAR(30) NOT NULL,  
    depart_time TIME,  
    arrival_time TIME,  
    source_stn  
    VARCHAR(20),  
    dest_stn  
    VARCHAR(20),  
    no_of_res_bogies  
    INT, bogie_capacity  
    INT
```

```
);  
CREATE TABLE passenger (  
    passenger_id INT  
    PRIMARY KEY,  
    passenger_name VARCHAR(20)  
    NOT NULL, address  
    VARCHAR(30),  
    age INT,  
    gender CHAR(1)
```

```
);  
CREATE TABLE ticket (  
    ticket_no INT  
    PRIMARY KEY,  
    train_no INT REFERENCES  
    train(train_no), passenger_id INT  
    REFERENCES passenger(passenger_id),  
    bogie_no INT,  
    no_of_berths  
    INT, tdate  
    DATE,  
    ticket_amt NUMERIC(10,2),  
    status CHAR(1) CHECK (status IN ('W','C'))
```

```
);  
INSERT INTO train VALUES  
(101,'Shatabdi Express','06:00','10:00','Pune','Mumbai',12,30),  
(102,'Koyana Express','07:30','14:00','Pune','Kolhapur',8,30),  
(103,'Rajdhani Express','20:00','06:00','Mumbai','Delhi',15,30);  
INSERT INTO passenger  
VALUES  
(1,'Mr.Raj','Pune',32,'M'),  
(2,'Riya','Mumbai',4,'F'),  
(3,'Suresh','Satara',65,'M'),
```



```

(4,'Anita','Pune',28,'F'),
(5,'Mahesh','Delhi',62,'M');
INSERT INTO ticket VALUES
(5001,101,1,1,2,'2019-12-
12',1200.00,'C'),
(5002,102,2,2,1,'2019-12-12',900.00,'W'),
(5003,103,3,1,2,'2019-12-12',4500.00,'C'),
(5004,103,5,3,1,'2019-12-12',4600.00,'W'),
(5005,101,4,5,1,'2021-04-12',1200.00,'C');
SELECT train_name, depart_time,
arrival_time FROM train;
SELECT
passenger_name,
address FROM
passenger
WHERE age > 60;
SELECT source_stn,
dest_stn FROM train
WHERE no_of_res_bogies > 10;
SELECT
passenger_name
FROM passenger
WHERE gender = 'M';
CREATE OR REPLACE FUNCTION
trg_train_bogie_capacity() RETURNS
TRIGGER AS $$
BEGIN
    IF NEW.bogie_capacity > 30 THEN
        RAISE EXCEPTION 'Bogie capacity cannot be
greater than 30'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
before_insert_train BEFORE
INSERT ON train
FOR EACH ROW
EXECUTE FUNCTION trg_train_bogie_capacity();
CREATE OR REPLACE FUNCTION
fn_total_ticket_amount_on_date(p_date DATE) RETURNS
NUMERIC(12,2) AS $$
DECLARE
    v_total
NUMERIC(12,2);
BEGIN
    SELECT
    COALESCE(SUM(ticket_amt),0
) INTO v_total
FROM ticket
WHERE tdate =
p_date; RETURN

```

```
        v_total;  
END;  
$$ LANGUAGE plpgsql;  
SELECT fn_total_ticket_amount_on_date('2019-12-12');
```

Output:

Query 1: SELECT train_name, depart_time, arrival_time FROM train

train_name	depart_time	arrival_time
Shatabdi Express	06:00	10:00
Koyana Express	07:30	14:00
Rajdhani Express	20:00	06:00

Query 2: SELECT passenger_name, address FROM passenger WHERE age > 60

passenger_name	address
Suresh	Satara
Mahesh	Delhi

Query 3: SELECT source_stn, dest_stn FROM train WHERE no_of_res_bogies > 10

source_stn	dest_stn
Pune	Mumbai
Mumbai	Delhi

Query 4: SELECT passenger_name FROM passenger WHERE gender = 'M'

passenger_name
Mr.Raj
Suresh
Mahesh

Query 5: SELECT fn_total_ticket_amount_on_date('2019-12-12')

fn_total_ticket_amount_on_date('2019-12-12')
11200

Assignment 33- Movie Actor Producer

Modern movie slip with producer list and Madhuri Dixit charges.

Commands :

```
DROP TABLE IF EXISTS movie_actor, movie, actor,
producer CASCADE; CREATE TABLE producer (
    producer_id INT PRIMARY
    KEY, pname VARCHAR(30)
    NOT NULL,
    p_address VARCHAR(30)
);
CREATE TABLE movie (
    movie_id INT
    PRIMARY KEY,
    m_name VARCHAR(30)
    NOT NULL,
    release_year
    INT, budget
    NUMERIC(12,2),
    producer_id INT REFERENCES producer(producer_id)
);
CREATE TABLE actor (
    actor_id INT PRIMARY
    KEY, a_name
    VARCHAR(30) NOT
    NULL, city
    VARCHAR(30)
);
CREATE TABLE movie_actor (
    movie_id INT REFERENCES
    movie(movie_id), actor_id INT
    REFERENCES actor(actor_id),
    charges NUMERIC(12,2),
    PRIMARY KEY (movie_id, actor_id)
);
INSERT INTO producer
VALUES (1,'Skyline
Films','Mumbai'),
(2,'Capital Arts','Delhi'),
(3,'Lotus Vision','Pune');
INSERT INTO movie
VALUES (101,'Future
Saga',2025,180.00,1),
(102,'Metro
Nights',2024,95.00,2),
(103,'Hero
Returns',2025,120.00,3);
INSERT INTO actor VALUES
(11,'Madhuri Dixit','Delhi'),
(12,'Amitabh
Bachchan','Mumbai'),
```

```

(13,'Rajkummar
Rao','Delhi'),
(14,'Vidya Balan','Pune');
INSERT INTO movie_actor
VALUES (101,11,20.00),
(102,11,16.00),
(103,12,18.00),
(103,13,9.00);
SELECT *
FROM movie
WHERE release_year = 2025;
SELECT
a_name
FROM
actor
WHERE city = 'Delhi';
SELECT m_name,
budget FROM
movie
WHERE budget > 100;
SELECT pname,
p_address FROM
producer;
CREATE OR REPLACE FUNCTION
trg_movie_year_check27() RETURNS
TRIGGER AS $$
BEGIN
    IF NEW.release_year > EXTRACT(YEAR FROM CURRENT_DATE) THEN
        RAISE EXCEPTION 'Release year should not be greater
        than current year'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER
before_insert_movie27 BEFORE
INSERT ON movie
FOR EACH ROW
EXECUTE FUNCTION trg_movie_year_check27();
CREATE OR REPLACE FUNCTION
fn_madhuri_movie_charges() RETURNS
TABLE (
    movie_name
    VARCHAR(30), charges
    NUMERIC(12,2)
) AS $$
DECLARE
    rec
    RECORD;
    cur
    CURSOR
    FOR

```

```
SELECT m.m_name,  
ma.charges FROM  
movie m  
JOIN movie_actor ma ON m.movie_id =  
ma.movie_id JOIN actor a ON  
a.actor_id = ma.actor_id  
WHERE a.a_name = 'Madhuri Dixit';  
  
BEGIN  
  OPEN  
  cur;  
  LOOP  
    FETCH cur INTO rec;  
    EXIT WHEN NOT FOUND;  
    movie_name :  
      rec.m_name;  
    charges : rec.charges;  
    RETURN NEXT;  
  END LOOP;  
  CLOSE cur;  
END;  
  
SELECT * FROM fn_madhuri_movie_charges();
```

Output:

Query 1: SELECT * FROM movie WHERE release_year = 2025

movie_id	m_name	release_year	budget	producer_id
101	Future Saga	2025	180	1
103	Hero Returns	2025	120	3

Query 2: SELECT a_name FROM actor WHERE city = 'Delhi'

a_name
Madhuri Dixit
Rajkummar Rao

Query 3: SELECT m_name, budget FROM movie WHERE budget > 100

m_name	budget
Future Saga	180
Hero Returns	120

Query 4: SELECT pname, p_address FROM producer

pname	p_address
Skyline Films	Mumbai
Capital Arts	Delhi
Lotus Vision	Pune

Query 5: SELECT * FROM fn_madhuri_movie_charges()

movie_name	charges
Future Saga	20
Metro Nights	16

Assignment 34- Student Course Enrollment

Student-course slip with age filter, B.C.A course query, function and view.

Commands :

```
DROP TABLE IF EXISTS enrollment, course,  
student CASCADE; CREATE TABLE student (  
    student_id INT  
    PRIMARY KEY, name  
    VARCHAR(30) NOT  
    NULL,  
    age INT  
);  
CREATE TABLE course (  
    course_id INT  
    PRIMARY KEY,  
    course_name VARCHAR(40)  
    NOT NULL, credits INT  
);  
CREATE TABLE enrollment (  
    student_id INT REFERENCES  
    student(student_id), course_id INT  
    REFERENCES course(course_id),  
    enrollment_date DATE,  
    PRIMARY KEY (student_id, course_id)  
);  
INSERT INTO student VALUES  
(1,'Pratik',21),  
(2,'Pooja',22),  
(3,'Ram',20),  
(4,'Sneha',19);  
INSERT INTO course VALUES  
(101,'B.Sc(Comp.Sci)',4),  
(102,'B.C.A',5),  
(103,'Statistics',3),  
(104,'AI Basics',4);  
INSERT INTO  
enrollment VALUES  
(1,102,'2024-06-10'),  
(2,101,'2024-06-12'),  
(3,102,'2024-06-14'),  
(4,104,'2024-06-16');  
INSERT INTO student VALUES  
(5,'Pallavi',23),  
(6,'Ritesh',20);  
SELECT *  
FROM course  
WHERE credits > 3;  
SELECT name  
FROM  
student  
WHERE age
```



```

> 20;
SELECT
s.student_id
FROM student
s
JOIN enrollment e ON s.student_id =
e.student_id JOIN course c ON
c.course_id = e.course_id
WHERE c.course_name = 'B.C.A';
CREATE OR REPLACE FUNCTION
fn_student_name_start_p() RETURNS TABLE
(
    student_id
    INT, name
    VARCHAR(30),
    age INT
) AS
$$
BEGIN
    RETURN QUERY
    SELECT student.student_id,
    student.name, student.age FROM
    student
    WHERE name ILIKE 'P%';
END;
$$ LANGUAGE plpgsql;
SELECT * FROM fn_student_name_start_p();
CREATE OR REPLACE VIEW
vw_student_course_names28 AS SELECT
s.name, c.course_name
FROM student s
JOIN enrollment e ON s.student_id =
e.student_id JOIN course c ON
c.course_id = e.course_id
ORDER BY s.name, c.course_name;
SELECT * FROM vw_student_course_names28;

```

Output:

Query 1: SELECT * FROM course WHERE credits > 3

course_id	course_name	credits
101	B.Sc(Comp.Sci)	4
102	B.C.A	5
104	AI Basics	4

Query 2: SELECT name FROM student WHERE age > 20

name
Pratik
Pooja
Pallavi

Query 3: SELECT s.student_id FROM student s JOIN enrollment e ON s.student_id = e.student_id JOIN course c ON c.course_id = e.course_id WHERE c.course_name = 'B.C.A'

student_id
1
3

Query 4: SELECT * FROM fn_student_name_start_p()

student_id	name	age
1	Pratik	21
2	Pooja	22
5	Pallavi	23

Query 5: SELECT * FROM vw_student_course_names28

name	course_name
Pooja	B.Sc(Comp.Sci)
Pratik	B.C.A
Ram	B.C.A
Sneha	AI Basics

Assignment 35- Bus Route Driver

Bus, Route and Driver with route function and Satara delete block.

Commands :

```
DROP TABLE IF EXISTS driver, bus, route

CASCADE; CREATE TABLE route (
    route_no INT
    PRIMARY KEY, source
    VARCHAR(20),
    destination
    VARCHAR(20),
    no_of_stations INT
);
CREATE TABLE bus (
    bus_no INT PRIMARY
    KEY, b_capacity INT
    NOT NULL,
    depot_name
    VARCHAR(20),
    route_no INT REFERENCES route(route_no)
);
CREATE TABLE driver
( driver_no INT
    PRIMARY KEY,
    driver_name VARCHAR(20)
    NOT NULL, license_no INT
    UNIQUE NOT NULL, address
    VARCHAR(20),
    d_age INT,
    salary NUMERIC(10,2),
    bus_no INT REFERENCES bus(bus_no)
);
INSERT INTO route VALUES
(1,'Delhi','Jaipur',12),
(2,'Delhi','Agra',10),
(3,'Pune','Mumbai',18),
(4,'Kothrud','Swargate',9);
INSERT INTO bus VALUES
(31,55,'Kothrud',1),
(32,60,'Pune Station',2),
(33,45,'Kothrud',3),
(34,70,'Kothrud',4);
INSERT INTO driver VALUES
(301,'Ajay',7101,'Pune',54,32000,
31),
(302,'Bharat',7102,'Satara',49,22000,33),
(303,'Chetan',7103,'Pune',61,28000,34),
(304,'Deepak',7104,'Nashik',52,26000,32),
(305,'Farhan',7105,'Pune',42,30000,31);
SELECT *
FROM bus
WHERE b_capacity > 50;
```

```

SELECT *
FROM route
WHERE source = 'Delhi';
SELECT driver_name,
license_no FROM driver
WHERE salary > 25000;
SELECT
d.driver_name
FROM driver d
JOIN bus b ON d.bus_no = b.bus_no
WHERE b.depot_name = 'Kothrud';
CREATE OR REPLACE FUNCTION
fn_buses_of_route(p_route_no INT) RETURNS TABLE (
    bus_no
    INT,
    capacity
    INT,
    depot_name VARCHAR(20)
) AS
$$
BEGIN
    RETURN QUERY
    SELECT bus.bus_no, bus.b_capacity,
    bus.depot_name FROM bus
    WHERE route_no = p_route_no;
END;
$$ LANGUAGE plpgsql;
SELECT * FROM fn_buses_of_route(4);
CREATE OR REPLACE FUNCTION
trg_prevent_delete_satara_driver() RETURNS TRIGGER
AS $$
BEGIN
    IF OLD.address = 'Satara' THEN
        RAISE EXCEPTION 'Deletion not allowed for
        drivers living in Satara'; END IF;
    RETURN OLD;
END;
CREATE TRIGGER
before_delete_driver29 BEFORE
DELETE ON driver
FOR EACH ROW
EXECUTE FUNCTION trg_prevent_delete_satara_driver();

```

Output:

Query 1: SELECT * FROM bus WHERE b_capacity > 50

bus_no	b_capacity	depot_name	route_no
31	55	Kothrud	1
32	60	Pune Station	2
34	70	Kothrud	4

Query 2: SELECT * FROM route WHERE source = 'Delhi'

route_no	source	destination	no_of_stations
1	Delhi	Jaipur	12
2	Delhi	Agra	10

Query 3: SELECT driver_name, license_no FROM driver WHERE salary > 25000

driver_name	license_no
Ajay	7101
Chetan	7103
Deepak	7104
Farhan	7105

Query 4: SELECT d.driver_name FROM driver d JOIN bus b ON d.bus_no = b.bus_no WHERE b.depot_name = 'Kothrud'

driver_name
Ajay
Bharat
Chetan
Farhan

Query 5: SELECT * FROM fn_buses_of_route(4)

bus_no	capacity	depot_name
34	70	Kothrud

Assignment 36- Branch Customer Loan Application

Final bank slip with less-than-required view and multi-branch function.

Commands :

```
DROP TABLE IF EXISTS loan_application, customer,
```

```
branch CASCADE; CREATE TABLE branch (  
    bid INT PRIMARY KEY,  
    brname VARCHAR(30) NOT NULL,  
    brcity VARCHAR(20)  
);
```

```
CREATE TABLE  
    customer ( cno  
    INT PRIMARY  
    KEY,  
    cname VARCHAR(20)  
    NOT NULL, caddr  
    VARCHAR(35),  
    city VARCHAR(20)  
);
```

```
CREATE TABLE  
    loan_application (  
    lno INT PRIMARY  
    KEY,  
    lamtrequired  
    NUMERIC(12,2),  
    lamtapproved  
    NUMERIC(12,2), ldate  
    DATE,  
    bid INT REFERENCES  
    branch(bid), cno INT  
    REFERENCES  
    customer(cno)  
);
```

```
INSERT INTO branch VALUES  
(1,'Deccan','Pune'),  
(2,'J.M.Road','Pune'),  
(3,'Aundh','Pune');
```

```
INSERT INTO customer  
VALUES  
(101,'Amit','Baner','Pune'),  
(102,'Pooja','Nigdi','Pune'),  
(103,'Rohit','Camp','Pune'),  
(104,'Anita','Aundh','Pune');
```

```
INSERT INTO loan_application  
VALUES  
(1001,500000,450000,'2024-01-  
10',1,101),  
(1002,400000,350000,'2024-01-11',2,102),  
(1003,300000,300000,'2024-01-12',2,103),  
(1004,700000,650000,'2024-01-13',2,104),  
(1005,600000,550000,'2024-01-14',1,102),  
(1006,200000,180000,'2024-01-15',3,101);
```

```

SELECT
c.cname
FROM
customer c
JOIN loan_application l ON c.cno = l.cno
WHERE l.lamtapproved >100000;
SELECT MAX(lamtapproved) AS
max_approved FROM
loan_application;
SELECT SUM(l.lamtapproved) AS
total_sanctioned FROM
loan_application l
JOIN branch b ON b.bid = l.bid
WHERE b.brname = 'Deccan';
SELECT COUNT(*) AS
jm_road_count FROM
loan_application l
JOIN branch b ON b.bid = l.bid
WHERE b.brname = 'J.M.Road';
CREATE OR REPLACE VIEW
vw_less_than_required_loan30 AS SELECT
c.*, l.lno, l.lamtrequired, l.lamtapproved
FROM customer c
JOIN loan_application l ON
c.cno = l.cno WHERE
l.lamtapproved <
l.lamtrequired;
SELECT * FROM vw_less_than_required_loan30;
CREATE OR REPLACE FUNCTION
fn_customers_with_multiple_branches30() RETURNS
TABLE (
    customer_name
    VARCHAR(20), branch_count
    BIGINT
) AS
$$
BEGIN
    RETURN QUERY
    SELECT c.cname, COUNT(DISTINCT l.bid)
    FROM customer c
    JOIN loan_application l ON
    c.cno = l.cno GROUP BY
    c.cname
    HAVING COUNT(DISTINCT l.bid) > 1;
END;
SELECT * FROM fn_customers_with_multiple_branches30();

```

Output:

Query 1: SELECT c.cname FROM customer c JOIN loan_application l ON c.cno = l.cno WHERE l.lamtapproved >= 100000

cname
Amit
Pooja
Rohit
Anita
Pooja
Amit

Query 2: SELECT MAX(lamtapproved) AS max_approved FROM loan_application

max_approved
650000

Query 3: SELECT SUM(l.lamtapproved) AS total_sanctioned FROM loan_application l JOIN branch b ON b.bid = l.bid WHERE b.brname = 'Deccan'

total_sanctioned
1000000

Query 4: SELECT COUNT(*) AS jm_road_count FROM loan_application l JOIN branch b ON b.bid = l.bid WHERE b.brname = 'J.M.Road'

jm_road_count
3

Query 5: SELECT * FROM vw_less_than_required_loan30

cno	cname	caddr	city	lno	lamtrequired	lamtapproved
101	Amit	Baner	Pune	1001	500000	450000
102	Pooja	Nigdi	Pune	1002	400000	350000
104	Anita	Aundh	Pune	1004	700000	650000
102	Pooja	Nigdi	Pune	1005	600000	550000
101	Amit	Baner	Pune	1006	200000	180000

Query 6: SELECT * FROM fn_customers_with_multiple_branches30()

customer_name	branch_count
Amit	2
Pooja	2